
Dense, Lexical, and Space-Filling-Curve Retrieval in a Personal Knowledge Base

Andrey Maltsev
Independent researcher
github.com/guanchzhou/hilbert-paper

Abstract

Personal knowledge bases retrieve notes for language models under a token budget, yet it is not known which design choices change whether the right note reaches the model and which only change cost. We test four hypotheses, stated before data collection, on one frozen corpus of 1,225 notes and 5,155 chunks with 817 development questions and 817 sealed held-out questions, using paired Wilcoxon and exact McNemar tests with Holm correction. Dense chunk retrieval exceeds lexical note retrieval by 0.498 in recall at 10, while chunk-level and note-level dense indexing do not differ. Under a 6,000-token budget, returning chunks keeps the relevant note for 0.755 of questions against 0.613 for whole notes, and no question favours whole notes.

We also built a Hilbert-curve key for this study and evaluated it as a candidate filter. Its best setting reaches a recall of 0.304 against 0.698 for exhaustive cosine and fails non-inferiority. We show that its first level is an eight-bit sign hash; after mean-centring the anisotropic embeddings, angular hashing theory predicts the observed question-to-answer collision rate (0.037 observed, 0.038 predicted) and implies that about 71 independent keys would be needed for 90% recall. Compared on the same vectors, pgvector’s HNSW index is faster than ArangoDB’s vector index at matched quality, and both engines return identical one-hop neighbourhoods. On held-out questions the dense chunk configuration reaches a recall at 10 of 0.725.

Two exploratory studies follow. None of the 8 methods tested for putting more relevant notes into the same token budget beat plain cosine order. In a full factorial of eight pipeline switches, packing the best sentences of the top 30 notes kept a relevant note for 0.897 of held-out questions against 0.752 for chunk packs, with fewer tokens, yet a pre-registered local judge found those packs sufficient to answer less often (0.713 against 0.793); the best-recall combination’s development gain did not replicate on held-out questions, which follow-up checks trace to low power and to incomplete relevance labels. Used as a map whose facet regions an agent intersects, the Hilbert key also failed a pre-registered cost rule.

1 Introduction

Retrieval-augmented use of language models has moved from web-scale collections to small personal ones: a few thousand notes, written by one person, searched by that person’s assistant. At this scale the classical constraints change. Exhaustive similarity search over every chunk is cheap, so the index structure matters less than in web search. What matters is whether the right note is among

Preprint.

the few passages a model is given, and how many tokens those passages cost, because the context window is the scarce resource and long contexts are used unevenly [1].

Practitioners face a set of design switches with little controlled evidence for small corpora: lexical or dense scoring [2], [3], note-level or chunk-level representation [4], [5], the unit of text returned to the model, and the index structure. This study adds one candidate of its own: a compact text key that places every vector on a Hilbert curve [6], [7], so that the key can be stored in an ordinary database column and a prefix range of keys can act as a candidate filter. To test whether such a key is a useful addition to knowledge management for AI systems, we implemented one, the `hk1` key in the `zig-hilbert` library [8], and evaluate it here.

This study measures those switches on one frozen personal corpus, under a protocol fixed in advance, and investigates the mechanisms behind each result. Five research questions are posed.

RQ1 Does dense retrieval find relevant notes that lexical retrieval misses, and the reverse?

RQ2 Is the chunk or the note the better unit of indexing for dense retrieval?

RQ3 Under a fixed token budget, which unit of returned text best preserves the relevant note?

RQ4 Can a prefix range of Hilbert keys replace exhaustive cosine search as a candidate filter, and if not, why not?

RQ5 At this corpus size and at forty times it, does query latency or a graph operation justify an engine other than Postgres, specifically ArangoDB?

The contributions are: a controlled comparison with pre-stated hypotheses, paired tests, and a sealed held-out set (Section 5); an analytical account of the Hilbert key as an eight-bit angular hash, validated against observation after correcting for embedding anisotropy (Section 6.2); a failure taxonomy of dense retrieval on this corpus (Section 6.3); and a fully reproducible artefact in which every number in the text is read from a result file (see Data and code availability).

2 Background and related work

2.1 Lexical and dense retrieval

BM25 scores a document by term frequency, inverse document frequency, and length normalisation [2]. Dense passage retrieval encodes the question and the passage separately and ranks by inner product [3]. Across the eighteen BEIR collections neither family dominates, and BM25 is a strong zero-shot baseline [9]. Hybrid retrieval combines the two, commonly by reciprocal rank fusion [10], and has been reported to be more robust than either alone [11]. Embedding models such as Qwen3-Embedding are trained with instruction prefixes for the query side [12].

2.2 Documents as sets of vectors

Late-interaction models score a passage by matching each query token to its best document token [4]. MUVIRA compresses a multi-vector document into a single fixed-dimensional encoding whose inner product approximates the set similarity [5]. A note represented by the mean of its chunk vectors is the simplest single-vector summary; a note scored by its best chunk is the simplest set score. RQ2 compares these two.

2.3 Approximate search, hashing, and space-filling curves

Graph indexes such as HNSW make exhaustive search unnecessary at large scale [13]; pgvector implements HNSW inside Postgres [14]. Inverted-file (IVF) indexes partition the vectors into lists around learned centroids and search only the lists nearest the query [15], [16]; the Faiss library implements them [17]. Quantisation-based methods protect the neighbours that matter [18], and redundant assignment recovers neighbours near partition boundaries [19]. Random-hyperplane hashing preserves angular similarity: two vectors at angle θ fall on the same side of a random hyperplane with probability $1 - \theta/\pi$ [20], and concatenating and repeating such hashes yields locality-sensitive hashing with tunable recall [21]. A Hilbert curve maps a grid to a line while preserving locality better than other simple curves [6], [7]. The `hk1` key built for this study composes a fixed sign projection, a quantiser, and an eight-dimensional Hilbert index [8].

2.4 Geometry of embedding spaces

Contextual embeddings are anisotropic: they occupy a narrow cone, so that unrelated texts have positive cosine similarity [22]. Removing the common mean direction is a standard correction [23]. Section 6.2 shows that this property governs the behaviour of a sign-projection key.

2.5 Evaluation methodology

Normalised discounted cumulative gain discounts rank i by $\log_2(i + 1)$ [24], and mean reciprocal rank, introduced for the TREC-8 question-answering track, averages the reciprocal rank of the first correct answer [25]. For paired comparisons of retrieval systems over the same questions, randomisation, bootstrap, and signed-rank tests agree closely, while the sign test and paired t -test can mislead [26]. This study uses the Wilcoxon signed-rank test [27] for graded per-question scores, the exact McNemar test [28] for binary outcomes, percentile bootstrap intervals [29], and Holm’s step-down correction for multiple tests [30]. A non-inferiority test asks whether a cheaper method is worse than a reference by no more than a margin fixed in advance, rather than whether the two differ [31]. Section 4.7 explains each procedure as applied here.

3 Hypotheses

The measurement plan (`plan.md` in the repository) was written and committed before any development cell was run. It fixed the corpus, the question set, the metrics, the budget, and the following confirmatory hypotheses with their acceptance criteria. Analyses not listed here are labelled exploratory throughout.

Table 1: Confirmatory hypotheses, family-wise $\alpha = 0.05$ with Holm correction [30]. Each test is described in Section 4.7.

Id	Hypothesis and acceptance criterion	Test
H1	Dense chunk retrieval has higher recall at 10 than lexical whole-note retrieval.	Wilcoxon [27], one-sided
H2	Dense chunk retrieval and dense note-mean retrieval differ in recall at 10.	Wilcoxon [27], two-sided
H3	At a 6,000-token budget, a chunk pack contains the relevant note more often than a whole-note pack. A cheaper unit that loses the note counts as a regression.	exact McNemar [28], one-sided
H4	The best Hilbert-key filter is non-inferior to exhaustive cosine: recall at 10 no more than 0.03 below it, with median candidates at most 20 percent of the keyed chunks.	larger p of Wilcoxon and paired t on shifted differences, one-sided [31]

Exploratory analyses: E1, AND against OR semantics for lexical queries; E2 and E5, reciprocal-rank-fusion hybrids against dense retrieval; E3, chunk against section packs; E4, the best lexical arm against dense retrieval; a random-subset control for the key filter; the hashing model and the mean-centring intervention of Section 6.2; and the failure taxonomy of Section 6.3. Exploratory tests are Holm-corrected within their own family.

4 Materials and methods

4.1 Corpus

The corpus is the author’s gbrain knowledge base [32] at the time of measurement: markdown notes on software engineering, infrastructure, research, and genealogy, mirrored from an Obsidian [33] vault into Postgres 16 [34]. Notes are split into chunks by gbrain’s chunker and each chunk is embedded with Qwen3-Embedding-0.6B [12] at 1024 dimensions, run locally; gbrain records the model as `ollama:qwen3-embedding-8k`, after its Ollama provider [35]. Question vectors for this study were computed through a local endpoint that serves the same model with MLX [36]. The note title is prefixed to each chunk before embedding, a minimal form of the chunk contextualisation proposed as contextual retrieval [37]. Table 2 and Figure 1 describe the corpus and the questions.

Table 2: Corpus and question set. Source: `bench/manifest.json`, `bench/investigation.json`.

Property	Value
Live notes	1,225
Chunks with a vector (stored)	5,155 (5,160)
Chunks per note, median (90th percentile)	2 (10)
Tokens per chunk, median (90th percentile)	572 (987)
Chunks with an <code>hk1</code> key	5,079
Development questions	817
Words per question, median (interquartile range)	8 (6 to 11)
Relevant notes per question, mean	1.44
Relevant slugs absent from the live corpus	16
Held-out questions	817

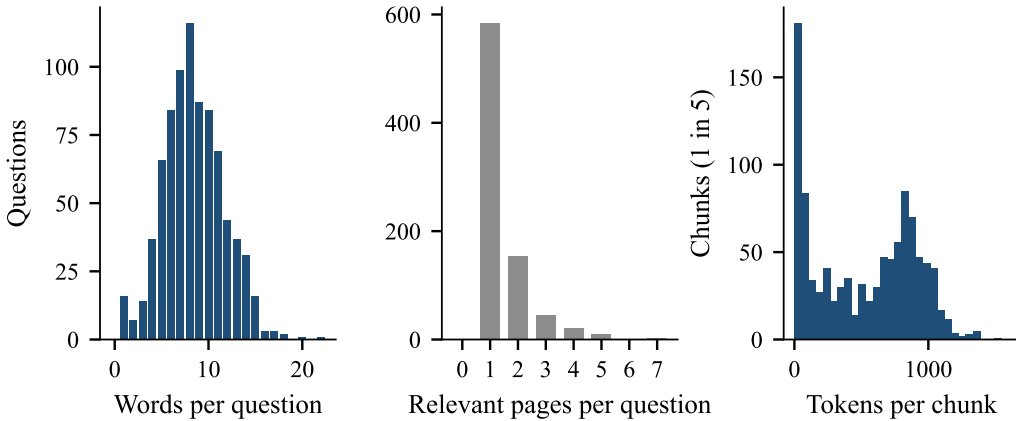


Figure 1: Distributions of question length, relevant notes per question, and chunk length (every fifth chunk). The bimodal chunk length reflects short structural chunks and long body chunks.

4.2 Questions and relevance

The question set is gbrain’s evaluation set: natural-language questions written against the brain, each with the slugs of the notes that answer it, split into a development half and a held-out half of 817 questions each. The held-out file was not read until every development analysis was frozen; it was then opened once, for the winning dense cell (Section 5.7). Relevance is binary and scored at note level after deduplication.

4.3 Retrieval conditions

Figure 2 shows the system. Each condition changes one factor.

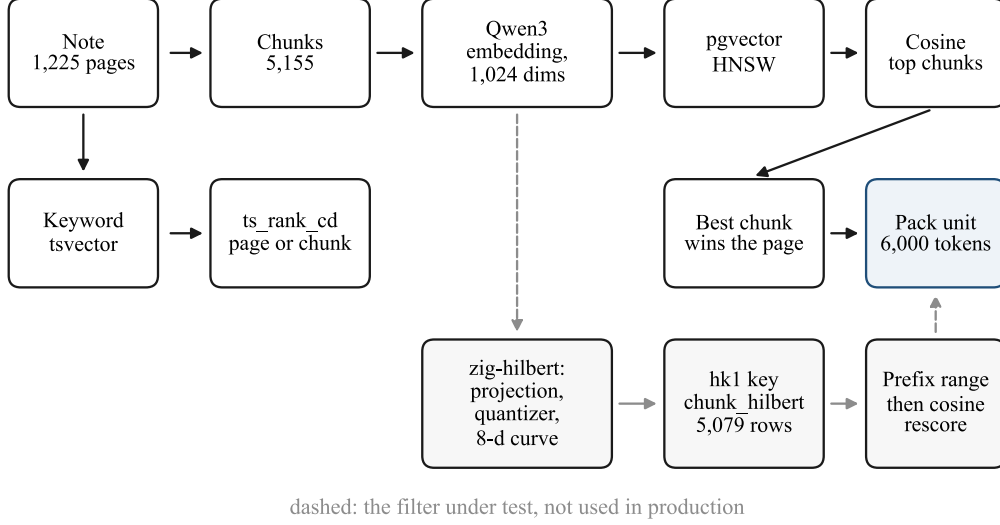


Figure 2: The pipeline. Solid arrows are the production path; the keyword path ranks by Postgres full-text search; the dashed path is the Hilbert-key filter under test.

- **Lexical, whole note (AND or OR).** Postgres full-text search over the note text [34]. The function `websearch_to_tsquery`, with the English configuration, stems the question’s words, drops stop words, and joins the remaining terms with AND; notes are ranked by `ts_rank_cd`, a cover-density score that rewards matched terms occurring close together [38]. The OR variant replaces conjunctions by disjunctions in the parsed query.
- **Lexical, chunk (AND or OR).** The same, over each chunk’s stored text-search vector; a note is ranked by its best chunk.
- **Dense, note mean.** Cosine between the question vector and the mean of the note’s chunk vectors.
- **Dense, chunk.** Cosine between the question vector and every chunk; a note is ranked by its best chunk.
- **Hybrid.** Reciprocal rank fusion [10] over the top 50 of a lexical arm and of dense chunk retrieval: a note at rank r in an arm receives $1/(60 + r)$ from that arm, the scores are summed, and the constant 60 limits the weight of the very top ranks.
- **Hilbert filter.** The probe of a question is the set of key ranges searched for it: the cell that contains the question’s own key at a chosen level, plus neighbouring cells (Section 4.4). Only chunks whose key falls in the probe ranges are rescored by exact cosine.

Question vectors were computed once with the live instruction prefix (Instruct: Given a web search query, retrieve relevant passages that answer the query, then Query: on a new line), the query format of Qwen3-Embedding [12], and reused by every condition. No condition calls a reranker.

4.4 The hk1 key

For a vector $e \in \mathbb{R}^{1024}$, zig-hilbert 0.2.1 computes the key in four steps [8]: (i) project onto eight axes $z_j = s_j \cdot e$, where $s_j \in \{-1, +1\}^{1024}$ are fixed by the seed 9e3779b97f4a7c15 (the 64-bit golden-ratio constant) through SplitMix64, a fast 64-bit pseudorandom generator [39]; (ii) divide by $\|e\|$; (iii) map each axis into 256 cells through the logistic function $\sigma(z) = 1/(1 + e^{-1.702z})$, which with the scale 1.702 stays within 0.01 of the standard normal distribution function [40], [41], so

that the cells would be nearly equally likely if the normalised projections were standard normal; (iv) index the eight cell coordinates on the eight-dimensional Hilbert curve with Skilling’s transpose algorithm [42], giving 64 bits written as `hk1:8:8:9e3779b97f4a7c15:` and sixteen hex digits, for example `hk1:8:8:9e3779b97f4a7c15:2e240e0214b885c0`. A probe level ℓ keeps the leading 8ℓ bits; a probe with r ranges adds up to $r - 1$ neighbouring cells across the nearest boundaries.

Proposition. The leading eight bits of an `hk1` key identify the orthant of $(z_1, \dots, z_8)$. Because $\sigma(z) \geq 1/2$ exactly when $z \geq 0$, the top bit of each cell coordinate is the sign of z_j , and the first level of the Hilbert curve is a bijection between the 2^8 sub-cubes and the leading eight index bits. A level-1 cell is therefore an eight-bit random-sign hash. If the projections behave as random hyperplanes, two unit vectors at angle θ share a level-1 cell with probability

$$P_1(\theta) = \left(1 - \frac{\theta}{\pi}\right)^8, \quad (1)$$

and a filter built from L independent such keys finds a neighbour with probability $1 - (1 - P_1)^L$, so that a target recall ρ requires

$$L = \frac{\ln(1 - \rho)}{\ln(1 - P_1)} \quad (2)$$

independent keys [20], [21]. The first claim was verified empirically: on every sampled pair, equality of the leading eight bits coincided with equality of the eight signs (Section 6.2).

4.5 Evidence packing

From the dense chunk ranking, four packs were built: the winning chunk; the chunk with one neighbour on each side (window); the chunk extended to the enclosing markdown heading (section); and the whole note (page). Units are walked in rank order and kept whole if they fit in the remaining 6,000 tokens, skipped otherwise. The budget is `gbrain`’s default delivery budget (`DEFAULT_RETURN_BUDGET` in its source), the amount of evidence it hands an agent per search; it was adopted as the operating point, not tuned, and Table 5 repeats the packing at other budgets. Tokens follow `gbrain`’s counter: one token per CJK (Chinese, Japanese, or Korean) character and one per four other characters.

4.6 Measures

For a question with relevant set R and a deduplicated ranked list L :

$$\text{R@10} = \frac{|R \cap L_{1..10}|}{|R|}, \quad (3)$$

$$\text{MRR} = \frac{1}{\min\{i : L_i \in R\}}, \quad (4)$$

$$\text{nDCG@10} = \frac{\sum_{i=1}^{10} \mathbb{1}[L_i \in R] / \log_2(i + 1)}{\sum_{i=1}^{\min(10, |R|)} 1 / \log_2(i + 1)}. \quad (5)$$

Recall at 10 is the share of a question’s relevant notes found among the first ten. Mean reciprocal rank (MRR) scores the first relevant note at rank i as $1/i$, and zero if none is returned [25]. Normalised discounted cumulative gain (nDCG) at 10 discounts each relevant note by the logarithm of its rank and divides by the best attainable value [24]. Survival is the indicator that a pack contains at least one relevant note. All figures are means over questions.

4.7 Statistical analysis

Every comparison is paired: both conditions answer the same questions, and each question serves as its own control. Per-question recall differences were tested with the Wilcoxon signed-rank test [27], a nonparametric test of whether paired differences are centred on zero: it ranks the absolute values of the non-zero differences and compares the rank sums of the positive and the negative ones. Zero differences were discarded. Survival, which is binary, was compared with the exact McNemar

test [28]. It uses only the discordant questions, those where exactly one of the two packs kept the relevant note, and asks whether they split evenly between the two packs; the exact form takes the p -value from the binomial distribution with probability one half.

Effect sizes are mean paired differences with 95 percent percentile bootstrap intervals [29]: questions were resampled with replacement 10,000 times, and the interval runs from the 2.5th to the 97.5th percentile of the resampled mean differences. For Wilcoxon comparisons we also report the matched-pairs rank-biserial correlation [43]: the rank sum of the positive differences minus the rank sum of the negative ones, divided by the total rank sum, with the ranks and discarded zeros of the test itself; it runs from -1 to 1 .

H4 is a non-inferiority hypothesis [31]. It does not ask whether the key filter differs from exhaustive cosine, but whether it is worse by more than a margin fixed in advance, 0.03 in recall at 10. The paired differences were shifted by adding the margin, and one-sided tests were applied to the shifted values; rejecting their null hypothesis would have shown the filter to be at most 0.03 worse. The shifted Wilcoxon test alone is unreliable when most paired differences are zero: the shift turns every tie into a small positive value, and those values can dominate the positive rank sum even for a clearly worse method. A one-sided paired t -test on the same shifted differences does not have this defect, so the reported p is the larger of the two. This can only make non-inferiority harder to show. The same defect was found and handled in the same way in the exploratory study of Section 7.

Holm’s step-down procedure [30] controls the family-wise error rate, the probability of at least one false rejection among several tests. The m p -values are sorted, the smallest is multiplied by m , the next by $m - 1$, and so on, and each adjusted value is capped at one and kept at least as large as the one before it. It was applied within the confirmatory family and, separately, within the exploratory family. Tests were computed with SciPy [44] and resampling with NumPy [45]. Latency is reported descriptively.

4.8 Controls

One factor changes per condition. The corpus is not re-embedded. Scratch databases for the scale experiment were separate databases, dropped in the step that recorded their numbers. The run-to-run stability of the dense cell was checked by re-embedding twenty questions five times (Section 5.7).

5 Results

5.1 RQ1: lexical and dense retrieval

Table 3: Primary retrieval conditions on 817 development questions.

Condition	R@10	95% CI	MRR	nDCG@10
Lexical, note, AND	0.200	0.174 to 0.226	0.180	0.175
Lexical, chunk, AND	0.138	0.116 to 0.161	0.138	0.128
Dense, note mean	0.683	0.654 to 0.713	0.497	0.521
Dense, chunk	0.698	0.669 to 0.727	0.517	0.538

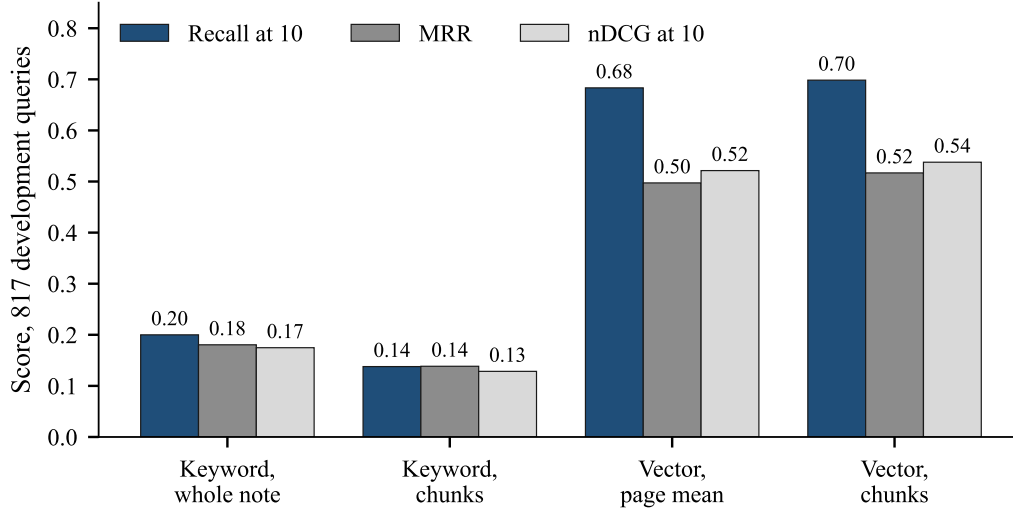


Figure 3: Recall at 10, MRR, and nDCG at 10 for the primary conditions.

Dense chunk retrieval reached a recall at 10 of 0.698 against 0.200 for lexical whole-note retrieval (Table 3). The paired difference was 0.498 (95% CI 0.463 to 0.534), matched-pairs rank-biserial correlation 0.91, Holm $p = 5.2 \times 10^{-80}$. H1 is supported. The advantage holds at every depth (Figure 4).

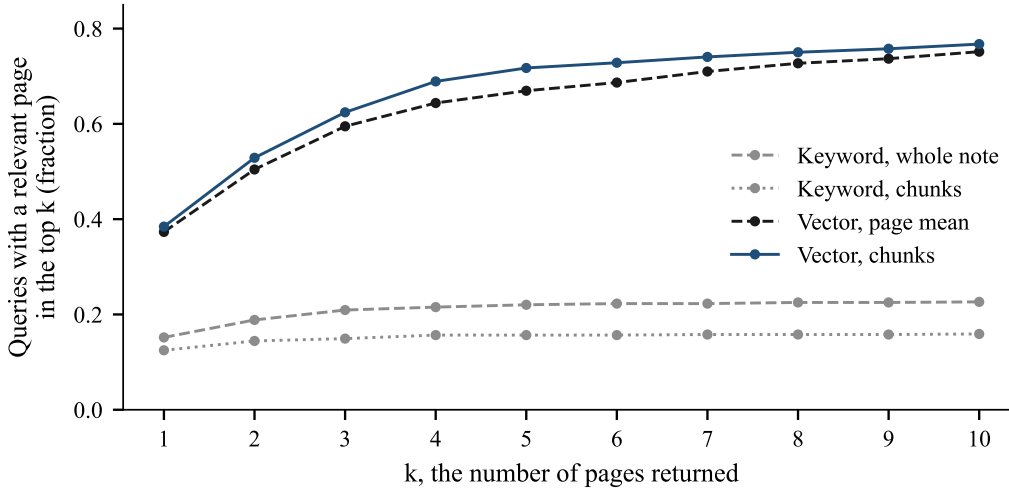


Figure 4: Share of questions with a relevant note in the top k .

Per question (Figure 5), both conditions found a relevant note for 163 questions, dense retrieval alone for 464, lexical retrieval alone for 22, and neither for 168. Lexical retrieval returned no note at all for 401 questions.

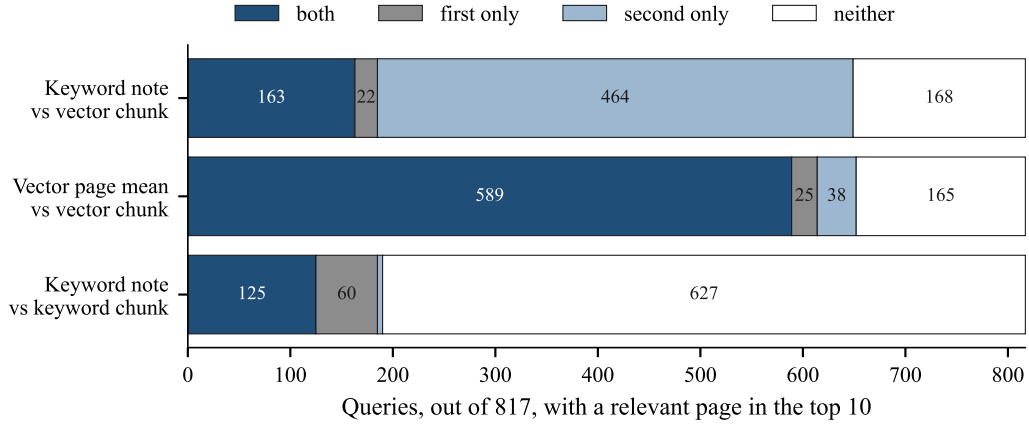


Figure 5: Per-question agreement between pairs of conditions on whether a relevant note is in the top 10.

5.2 RQ2: note or chunk

Dense chunk retrieval and dense note-mean retrieval reached 0.698 and 0.683. The difference was 0.015 (95% CI -0.003 to 0.034), Holm $p = 0.284$. H2 is not supported: at this corpus size the two representations rank notes equally well. The two disagreed on only 63 questions.

5.3 RQ3: the evidence unit under a budget

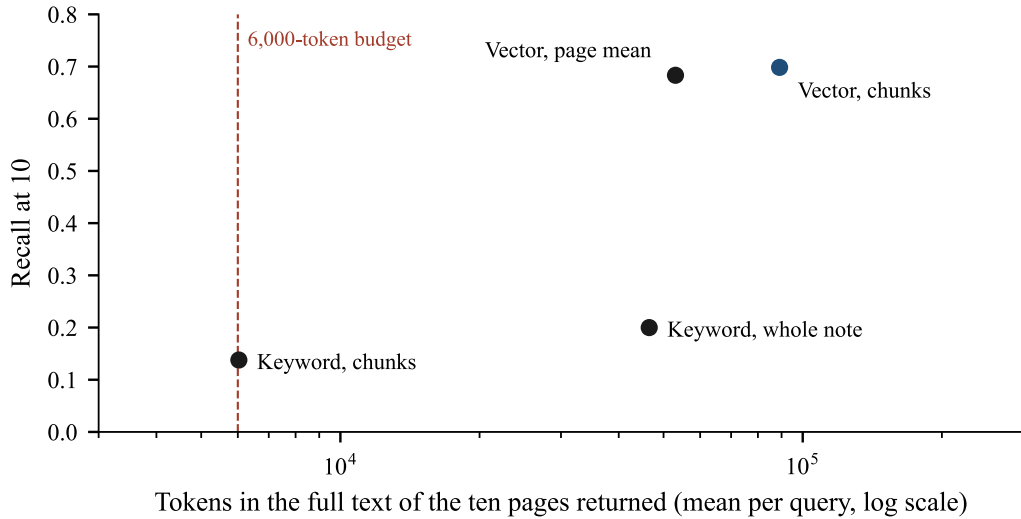


Figure 6: Recall at 10 against the tokens in the full text of the ten notes returned (mean, log scale). The 6,000-token budget is marked.

The ten notes returned by dense chunk retrieval contain on average 89,120 tokens, about fifteen times the budget (Figure 6), so every pack must select.

Table 4: Evidence units packed from the same ranking under a 6,000-token budget. Means per question.

Unit	Tokens	Hits skipped	Survival	95% CI
chunk	5,503	1.42	0.755	0.726 to 0.783
window	5,661	4.85	0.687	0.654 to 0.717
section	5,648	2.36	0.739	0.709 to 0.769
page	5,429	4.89	0.613	0.579 to 0.646

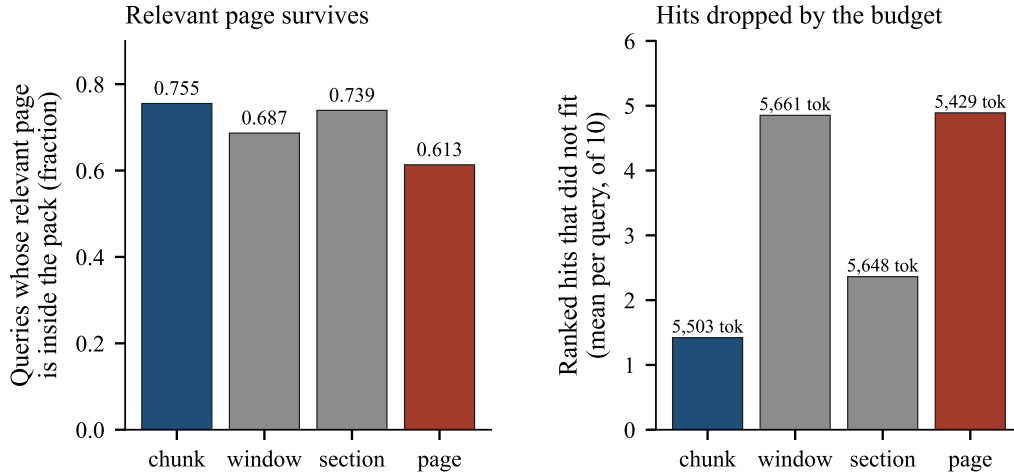


Figure 7: Survival of the relevant note, and ranked hits skipped by the budget, per unit.

All four packs spend nearly the whole budget (Table 4); they differ in how many distinct notes fit. The chunk pack kept the relevant note in 0.755 of questions and the page pack in 0.613: a difference of 0.142 (95% CI 0.119 to 0.166). Of the discordant questions, 116 favoured the chunk pack and 0 the page pack; exact McNemar Holm $p = 3.6 \times 10^{-35}$. H3 is supported. The upper bound for any pack is the share of questions with a relevant note in the top ten, 0.767; the chunk pack attains 98 percent of it.

Other budgets. The same packing at 1,000, 2,000, 4,000, 6,000, 12,000 and 24,000 tokens (Table 5) gives the same order at every budget: chunks keep the relevant note at least as often as any other unit, and whole notes never more often. The gap is largest when the budget is small: at 1,000 tokens chunks keep it for 0.410 of questions and whole notes for 0.157. At large budgets all units approach the ceiling set by the ten notes walked, and whole notes remain lowest while delivering the most text: at 24,000 tokens, 0.737 in 17,733 tokens against 0.767 in 6,655 for chunks. This sweep was run after the hypotheses were tested and is exploratory.

Table 5: Share of the 817 development questions whose relevant note is in the pack, by token budget and evidence unit (top 10 notes of the dense chunk ranking, units kept whole if they fit). 6,000 is gbrain’s default. Source: bench/budget-sweep.json.

Budget	Chunk	Window	Section	Whole note
1,000	0.410	0.165	0.366	0.157
2,000	0.596	0.399	0.562	0.367
4,000	0.727	0.589	0.700	0.540
6,000	0.755	0.688	0.739	0.612
12,000	0.767	0.754	0.754	0.683
24,000	0.767	0.766	0.767	0.737

5.4 RQ4: the Hilbert-key filter

Table 6: All sixteen probe settings on chunk keys. Last row: exhaustive cosine. Range time is the Postgres range count per question on the live key table.

Level	Ranges	Median cand.	R@10	nDCG@10	Range ms/q.
0	1	5,079	0.699	0.539	1.821
0	4	5,079	0.699	0.539	1.781
0	8	5,079	0.699	0.539	2.054
0	16	5,079	0.699	0.539	1.852
1	1	24	0.070	0.062	0.020
1	4	100	0.163	0.141	0.058
1	8	188	0.241	0.206	0.122
1	16	329	0.304	0.262	0.123
2	1	0	0.000	0.001	0.012
2	4	0	0.000	0.001	0.041
2	8	1	0.004	0.004	0.038
2	16	1	0.006	0.006	0.069
3	1	0	0.000	0.000	0.010
3	4	0	0.000	0.000	0.019
3	8	0	0.000	0.001	0.037
3	16	0	0.000	0.001	0.049
—	—	5,155	0.698	0.538	—

Level 0 covers the whole key space and equals exhaustive search. Levels 2 and 3 return almost no candidates, because 2^{16} and 2^{24} cells hold 5,079 chunks. Level 1 is the informative level (Table 6, Figure 8). Its best setting, sixteen ranges, read a median of 329 chunks and reached 0.304; the difference from exhaustive cosine was -0.394 (95% CI -0.427 to -0.361), non-inferiority Holm $p = 1.000$. H_4 is rejected. The paired t -test on the shifted differences was added after this run and computed in a separate run of the same script on the corpus as it stood then, 5,159 chunks against 5,157 in the main run: the difference was -0.400 (95% CI -0.433 to -0.366), with $p = 1.000$ for the shifted Wilcoxon test and $p = 1.000$ for the paired t -test. The conclusion is unchanged.

Table 7: Exploratory control: uniformly random subsets of keyed chunks of the same size, rescored by cosine, mean of three seeds.

Candidates	Random subset R@10	hk1 level 1 R@10	Ratio
24	0.025	0.070	2.8
100	0.084	0.163	1.9
188	0.111	0.241	2.2
329	0.172	0.304	1.8
1,000	0.352	—	—
2,500	0.547	—	—

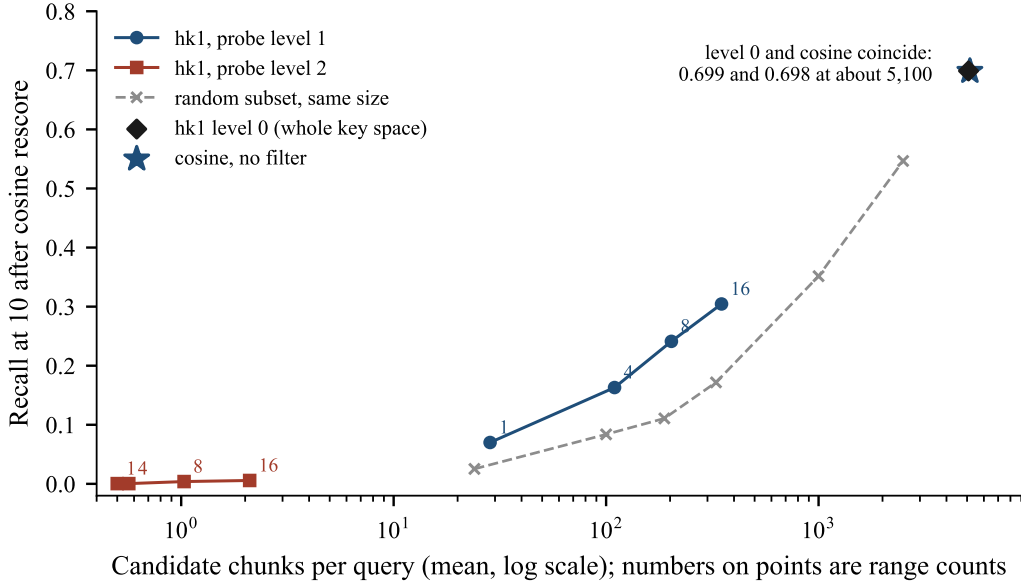


Figure 8: Recall at 10 after rescore against candidates per question, for level-1 and level-2 probes, the random control, level 0, and exhaustive cosine.

At equal candidate counts the level-1 filter recalls 1.8 to 2.8 times as much as a random subset (Table 7): the key carries angular information. Shared prefix length rises with cosine (Figure 9), but a chunk’s true nearest neighbour, at median cosine 0.83, shares the first eight bits in only 21 percent of cases. Section 6.2 explains why.

Efficiency per token. An index is judged here by what it delivers per token, on two cost axes. The first is the tokens processed: the text of the candidate chunks. The second is the tokens delivered: the 6,000-token chunk pack the model reads, built by the rule of Section 5.3 (Table 8). With no filter the pack reproduces the survival of 0.755 reported there.

Per token delivered, the filter halves efficiency. With sixteen ranges a relevant note survives in 0.357 of packs, 0.068 per 1,000 delivered tokens, against 0.137 for cosine over every chunk. The pack fills to almost the same size either way (5,294 against 5,505 tokens), so the filter changes which notes fill it, not how much the model reads.

Per token processed, the order reverses. The filter reads about 1/16 of the text and keeps a relevant note at 0.216 per 100,000 processed tokens, against 0.028. Against random subsets of the same size for each question, hk1 is 1.8 to 2.6 times as efficient on both axes, so the key carries information on either measure.

Which axis applies depends on the reader. When a vector search ranks the candidates and only the pack is read, the delivered axis applies and the filter loses. When something has to read the candidates

themselves, the processed axis applies and the filter wins against reading everything. These figures were computed after the hypotheses were tested, on the corpus as it stood on 6 October (5,164 chunks, 4,969 keyed), and are exploratory; the pre-registered factorial study measures the same pack per filter.

Table 8: Efficiency of hk1 level 1 per token. Survival: a relevant note is in the 6,000-token chunk pack. Delivered: tokens in that pack; processed: tokens in the candidate chunks; both are means per question. Random rows use, for each question, as many keyed chunks as the hk1 setting returned, averaged over three seeds. Source: `bench/hk1-efficiency.json`.

Condition	Candidates	Survival	Delivered	Per 1k delivered	Processed	Per 100k processed
cosine, every chunk	5,164	0.755	5,505	0.137	2,673,514	0.028
cosine, keyed chunks	4,969	0.748	5,488	0.136	2,519,020	0.030
hk1 level 1, 1 ranges	28	0.092	4,456	0.021	13,939	0.659
random, same size as hk1 1 ranges	28	0.039	4,887	0.008	14,060	0.276
hk1 level 1, 4 ranges	107	0.196	5,134	0.038	52,663	0.372
random, same size as hk1 4 ranges	107	0.110	5,449	0.020	54,276	0.202
hk1 level 1, 8 ranges	198	0.295	5,204	0.057	96,698	0.305
random, same size as hk1 8 ranges	198	0.156	5,477	0.029	100,528	0.155
hk1 level 1, 16 ranges	342	0.357	5,294	0.068	165,097	0.216
random, same size as hk1 16 ranges	342	0.201	5,480	0.037	172,971	0.116

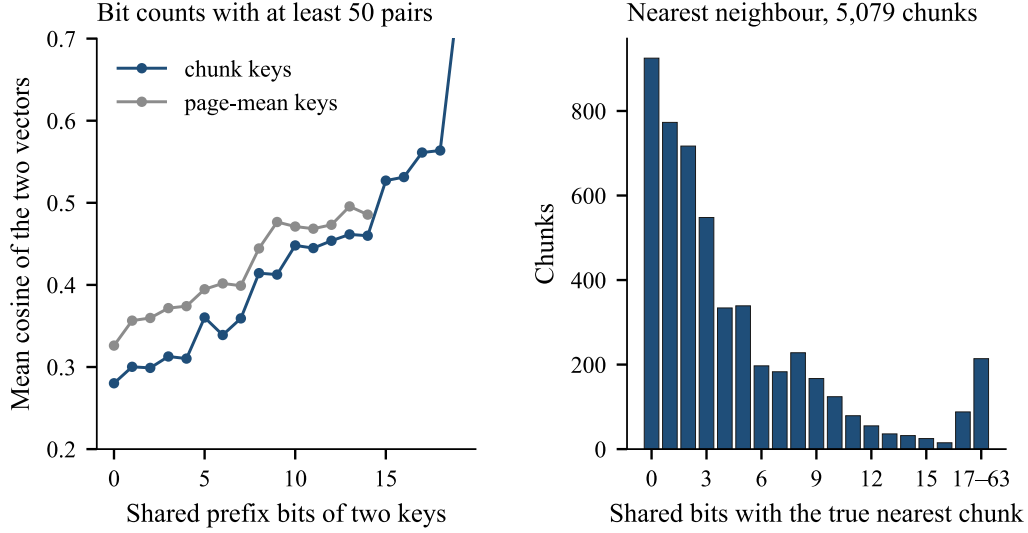


Figure 9: Left: mean pair cosine against shared prefix bits. Right: shared bits between each chunk and its true nearest neighbour; 214 duplicated chunks share all 64.

5.5 RQ5: engines

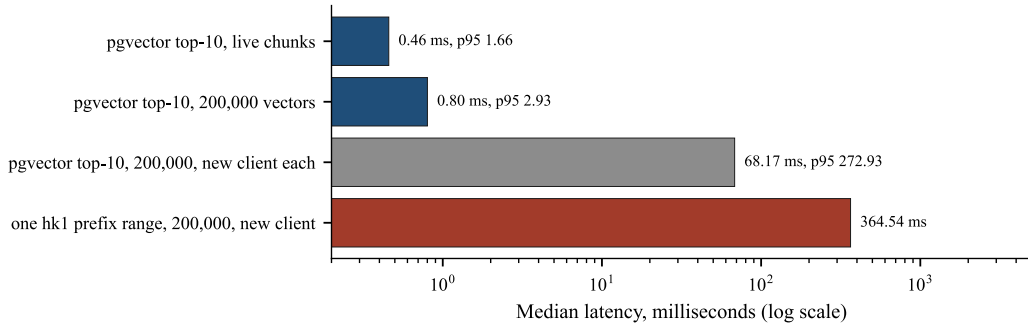


Figure 10: Median latency of a pgvector top-10 and of one hk1 prefix range. Grey and red bars include starting a client per query.

An HNSW top-10 in pgvector took 0.46 ms median (95th percentile 1.66 ms) on the live chunks and 0.80 ms (2.93 ms) on a scratch set of 200,000 vectors, inside one session (Figure 10). The 200,000-vector index occupied 1637 MB and built in 49.5 s. The one hk1 range timed at that scale, 365 ms, was a sequential scan because the query used the default collation; it is an upper bound. Latency does not motivate leaving Postgres at this size or at forty times it.

Choice of comparison engine. RQ5 asks whether query latency or a graph operation justifies an engine other than Postgres, so the comparison engine had to support both operations the knowledge base performs: nearest-neighbour search over chunk vectors and expansion along the links between notes. ArangoDB stores documents and a graph of edges natively, queries both in one language, and since version 3.12 has a vector index [46]. It is also the engine the author already runs on the same machine for a genealogy database, where it was chosen for documents and a graph together, which makes it a realistic alternative rather than a hypothetical one. Three other stores were considered and not measured. SQLite with the sqlite-vec extension [47] and libSQL, Turso’s fork of SQLite with a native vector type and an approximate vector index [48], are embedded single-file stores with the same relational model as Postgres: a one-hop expansion is the same SQL join, so they would answer a question about deployment, one file against a server, rather than about the graph operation. gbrain’s storage layer supports only Postgres and its WebAssembly build, PGLite [32], so neither

could have served the live corpus without a port. MongoDB was set aside because its vector search was taken to require the hosted Atlas service. That premise is out of date: vector search is available for the self-managed Community Edition through a separate search process, mongot, that requires a replica set [49]. All three were measured afterwards, in throwaway instances, with the same vectors, links and questions (Table 10).

Engine comparison. ArangoDB 3.12.10-1 [46] was run in a separate, throwaway container on its own port, with no persistent volume; the genealogy instance on the same machine was never started or contacted. The 5,156 live chunk vectors loaded in 1.8 s, and its vector index built in 0.8 s. ArangoDB’s vector index is a Faiss inverted-file (IVF) index [16], [17], [46]: it partitions the vectors into nLists lists around learned centroids, here 72 lists with the cosine metric, and a query searches only the nProbe lists whose centroids are nearest to it. Both engines answered the 817 development questions through one persistent client connection, fetching the top 50 chunks; quality is the share of the exact top ten chunks found, and note-level recall at 10 after collapsing chunks to notes (Table 9, Figure 11).

Table 9: ArangoDB against pgvector on the same 5,156 vectors and 817 questions. Client-observed latency over one persistent connection; ArangoDB is reached over HTTP with JSON, Postgres over its wire protocol. Source: bench/arango-vector.json.

Engine and setting	Median ms	p95 ms	Exact top-10 found	R@10
ArangoDB, nProbe 1	2.00	4.23	0.553	0.473
ArangoDB, nProbe 4	3.79	4.86	0.831	0.621
ArangoDB, nProbe 8	4.33	5.38	0.921	0.669
ArangoDB, nProbe 16	4.60	5.71	0.969	0.689
ArangoDB, nProbe 72	4.64	6.48	1.000	0.698
ArangoDB, full scan	12.62	14.05	1.000	0.698
pgvector HNSW, ef_search 50	2.10	2.92	0.986	0.698
pgvector HNSW, ef_search 100	2.16	3.16	0.994	0.699
pgvector HNSW, ef_search 200	1.88	3.44	0.999	0.697
pgvector HNSW, ef_search 400	2.43	4.26	1.000	0.698
pgvector, full scan	8.44	9.20	1.000	0.698

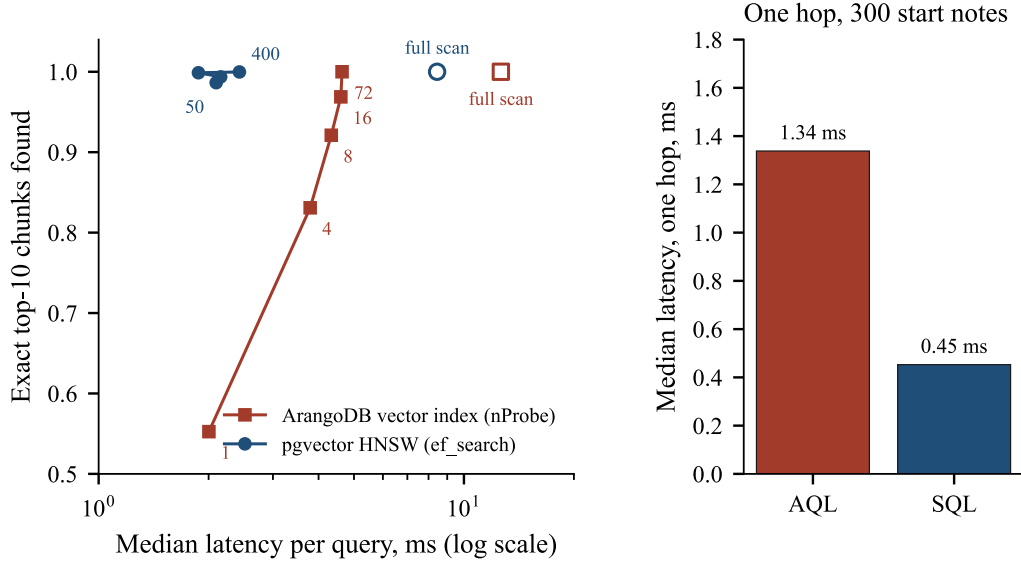


Figure 11: Left: share of the exact top-10 chunks found against median latency, for ArangoDB’s vector index at increasing nProbe (lists searched) and pgvector’s HNSW at increasing ef_search (size of the candidate list kept during the graph search [14]); open markers are full scans. Right: one-hop link expansion with hubs capped, in AQL and SQL.

At matched quality pgvector was faster. Its HNSW index [13], whose search effort is set by ef_search, the size of the candidate list kept while walking the graph [14], found 0.986 of the exact top-10 chunks at 2.10 ms median, and 0.9998 at 2.43 ms. ArangoDB reached 0.969 at 4.60 ms, and exact results only by probing all lists, at 4.64 ms. Full scans took 12.6 ms in ArangoDB and 8.4 ms in Postgres. Note-level recall is the same 0.698 in every exact configuration, so neither engine changes what is retrieved, only how fast.

For the graph operation, the 7,635 links between 1,225 live notes were loaded as an edge collection, and one-hop neighbours of 300 random linked notes were expanded with hubs of more than 50 neighbours excluded (17 notes). AQL, ArangoDB’s query language [46], and SQL returned identical neighbour sets for all 300 starts (mean Jaccard similarity, the size of the intersection over the size of the union [50], 1.00); the median time was 1.34 ms in AQL and 0.45 ms in SQL. The throwaway database was dropped and the container deleted after the run. A repeat after upgrading to PostgreSQL 18.6, pgvector 0.8.7 and ArangoDB 3.12.12 kept the same ordering (Appendix C).

Three more engines. SQLite 3.50.4 with sqlite-vec v0.1.9 [47], libSQL [48] (SQLite 3.45.1) with its approximate vector index, and MongoDB Community 9.0.2 with mongot [49] were each loaded with the live chunk vectors and links and asked the same 817 questions, with pgvector re-measured in the same session (Table 10). SQLite and libSQL run inside the client process, so their latency contains no network round trip and is not comparable with the server engines; MongoDB, a single-node replica set in a container beside its mongot, is. Every engine returned the same note-level recall within a thousandth, and every one returned the true one-hop neighbourhood for all 300 starts. MongoDB’s vector index answered in 9.1 to 10.6 ms against 1.5 ms for pgvector’s HNSW, and its one-hop expansion took 0.63 ms against 0.14 ms in SQL. The containers were removed after the run; the genealogy instance was not touched.

Table 10: The remaining engines on the same live vectors and 817 questions, with pgvector re-measured in the same session. One-hop expansion, median ms: SQL in Postgres 0.14, SQLite 0.009, libSQL 0.012, MongoDB 0.63; identical neighbour sets throughout. Source: bench/engines-d12.json.

Engine and setting	Median ms	p95 ms	Exact top-10 found	R@10
pgvector HNSW, ef_search 100	1.47	2.02	0.994	0.697
pgvector, full scan	13.47	17.12	1.000	0.698
SQLite + sqlite-vec, exact scan (in process)	5.89	6.21	1.000	0.698
libSQL vector index (in process)	8.79	9.35	1.000	0.698
libSQL, full scan (in process)	11.22	12.55	1.000	0.698
MongoDB vectorSearch, numCandidates 100	9.07	12.02	0.996	0.698
MongoDB vectorSearch, numCandidates 200	10.02	13.57	0.998	0.698
MongoDB vectorSearch, numCandidates 500	10.57	14.55	0.999	0.698
MongoDB vectorSearch, exact	10.62	15.23	1.000	0.698

On these measurements a second engine adds no capability Postgres lacks at this scale, and is slower at both operations tested.

5.6 Summary of hypothesis tests

Table 11: All tests. H rows are confirmatory, E rows exploratory; each family is Holm-corrected separately. Differences are in recall at 10, or in survival share for H3 and E3.

Id	Comparison	Difference (95% CI)	Holm p	Outcome
H1	Dense chunk vs lexical note (AND)	0.498 (0.463 to 0.534)	5.2×10^{-80}	supported
H2	Dense chunk vs dense note mean	0.015 (−0.003 to 0.034)	0.284	not supported
H3	Chunk pack vs page pack (survival)	0.142 (0.119 to 0.166)	3.6×10^{-35}	supported
H4	hk1 level 1, 16 ranges vs dense chunk (non-inferiority)	−0.394 (−0.427 to −0.361)	1.000	not supported
E1	Lexical note, OR vs AND	0.061 (0.028 to 0.093)	0.003	differs
E2	RRF hybrid (note lexical) vs dense chunk	−0.081 (−0.107 to −0.055)	8.6×10^{-8}	differs
E3	Chunk pack vs section pack (survival)	0.016 (0.007 to 0.026)	0.003	differs
E4	Lexical chunk OR vs dense chunk	−0.164 (−0.196 to −0.133)	7.0×10^{-20}	differs
E5	RRF hybrid (chunk lexical) vs dense chunk	0.003 (−0.019 to 0.025)	0.737	no difference

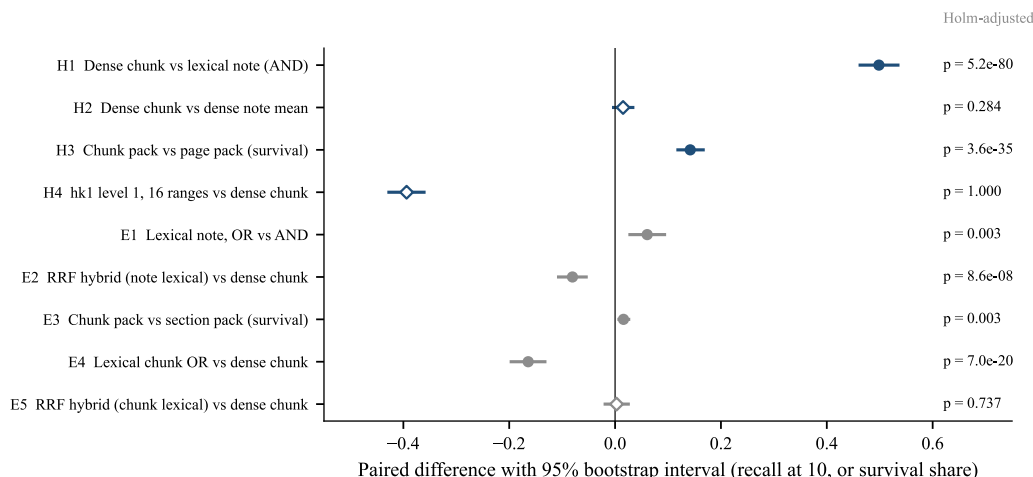


Figure 12: Paired differences with 95 percent bootstrap intervals. Filled markers: Holm-adjusted $p < 0.05$.

5.7 Generalisation and stability

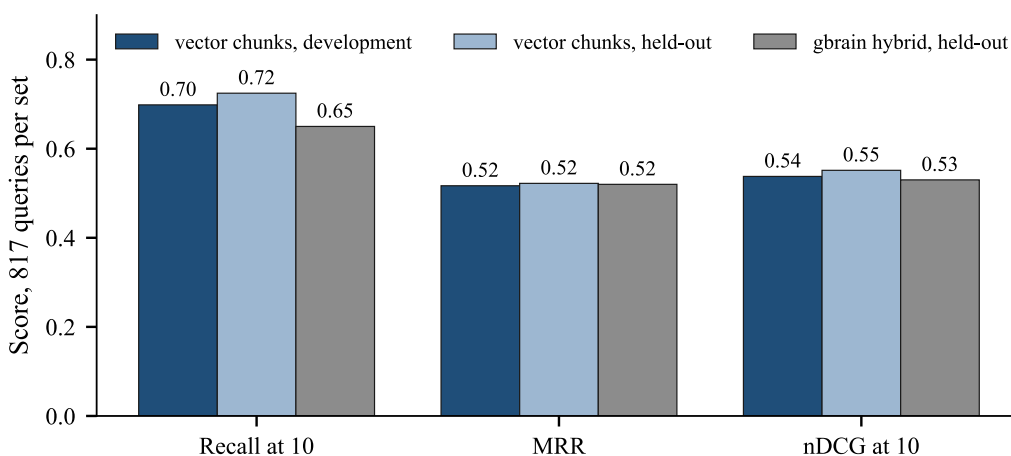


Figure 13: Dense chunk retrieval on development and held-out sets, with gbrain's production hybrid on the held-out set.

On the held-out questions the dense chunk cell reached recall at 10 of 0.725, MRR 0.522, and nDCG at 10 of 0.551, within the development interval's upper range (Figure 13). gbrain's production hybrid, measured on the same held-out set by gbrain's own evaluator, reached 0.65, 0.52, and 0.53. Re-embedding twenty questions five times changed no vector component by more than 0.00034 and changed no ranking; sampling over questions, captured by the bootstrap intervals, is the relevant uncertainty.

6 Mechanism analyses

6.1 Why lexical retrieval underperforms

websearch_to_tsquery joins every term with a conjunction, and the median question has 8 words. Replacing conjunctions by disjunctions (E1) raised whole-note lexical recall by 0.061 (95% CI 0.028 to 0.093), and chunk-level lexical recall from 0.138 to 0.534 (Figure 14). The best lexical arm still trailed dense retrieval by 0.164 (E4).

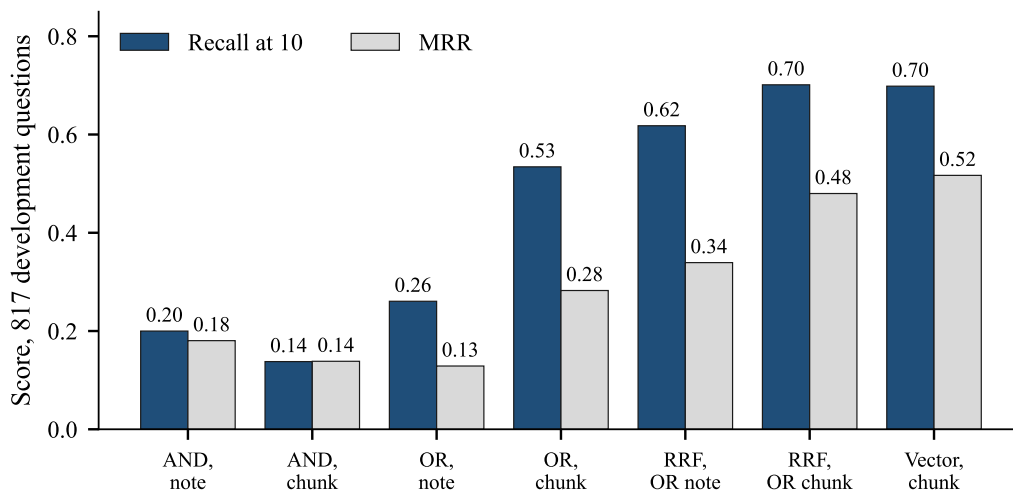


Figure 14: Lexical variants, reciprocal-rank-fusion hybrids, and dense chunk retrieval. Source: `bench/investigation.json`.

Fusing a lexical arm with dense retrieval did not help. With the whole-note lexical arm, fusion lowered recall by 0.081 (E2). With the stronger chunk-level arm, recall was unchanged, 0.003 (95% CI -0.019 to 0.025), while MRR fell from 0.517 to 0.480. On this corpus the lexical arm adds no notes that dense retrieval misses often enough to offset the rank noise it introduces. This is consistent with the production hybrid scoring below plain dense retrieval on held-out questions.

6.2 Why the Hilbert key is a weak filter

The Proposition of [Section 4.4](#) predicts the level-1 collision rate from the pair angle alone. Tested on 200,000 random chunk pairs, the leading-eight-bit equality and the eight-sign equality coincided on every pair, confirming that a level-1 cell is a sign hash. The magnitude, however, deviated: on raw vectors the observed collision rate was 0.0074 against a predicted 0.0190, and the deviation grew at low cosine ([Figure 15](#), left).

Anisotropy. The mean of the unit chunk vectors has norm 0.546: the embeddings occupy a narrow cone [\[22\]](#). Each projection then carries a large common offset, and the eight axes are unbalanced, with between 43 and 76 percent of chunks on the positive side. Per-axis sign agreement between two chunks, 0.532, is then no higher than for two unrelated chunks with the same marginals, although the random-hyperplane model expects 0.597. Cell occupancy is correspondingly skewed: the largest of the 256 cells holds 177 chunks against a uniform 19.8 ([Figure 16](#)).

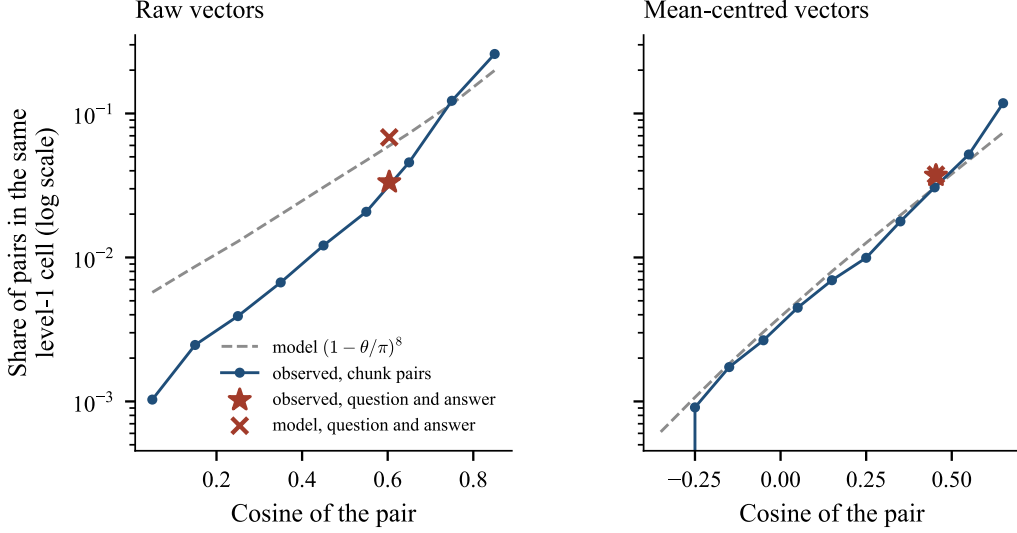


Figure 15: Share of pairs in the same level-1 cell against pair cosine, observed and predicted by Equation 1, for raw and mean-centred vectors. Stars and crosses: question-to-answer pairs at their median cosine.

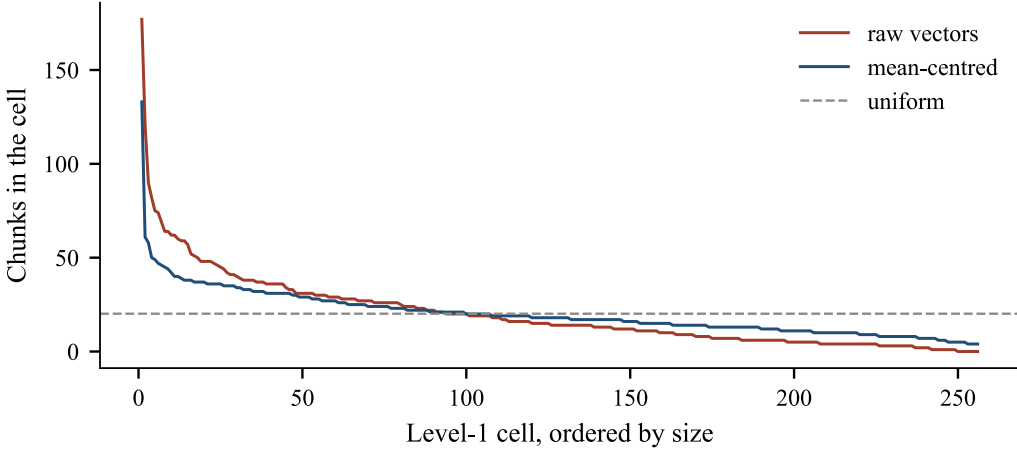


Figure 16: Chunks per level-1 cell, sorted, for raw and mean-centred vectors.

Intervention. Subtracting the corpus mean before keying, as in [23], balanced every axis (49 to 51 percent), filled all 256 cells, and brought the model into agreement with observation: per-axis agreement 0.500 against 0.500 predicted, cell collisions 0.0050 against 0.0054 (Figure 15, right). For question-to-answer pairs, with median centred cosine 0.45, the observed same-cell rate was 0.0370 and the predicted 0.0376.

Consequence. Centring corrected the geometry but did not rescue the filter: at equal candidate counts its recall curve lies only slightly above the raw curve (Figure 17). The limit is the angle between a question and its answer. At the median centred cosine, Equation 1 gives a collision probability of 0.032, and Equation 2 then requires about 71 independent eight-bit keys to find the answer in 90 percent of questions. One Hilbert key is one table. Probes across neighbouring cells recover part of the loss, which is why sixteen ranges reach 0.304, but a single locality-preserving key cannot approach exhaustive recall on questions whose answers are this far away in angle.

Many keys (exploratory). To test Equation 2 directly, zig-hilbert 0.3.0 added a second key type, hk2, that stores many independent eight-bit sign keys, one per table, with optional query-directed

probes [51], and keeps `hk1` unchanged. On the centred vectors of the 810 question-to-answer pairs, the share of pairs that shared a bucket in at least one of L tables followed the model at every table count: 0.412 observed against 0.418 predicted at 16 tables, 0.836 against 0.831 at 71, and 0.916 against 0.927 at 128. The figure of about 71 tables was optimistic because it used the median angle; averaged over each pair’s own angle, 90 percent is reached between 96 and 128 tables. The cost is the candidate set: at 128 tables a question shares a bucket with 40 percent of all chunks, and with four probes per table, 32 tables recover the exhaustive recall at 10 (0.696 against 0.698) only while scanning 41 percent of them. The theory holds; at this angle between questions and answers, hashing saves little over reading every vector.

A filter that keeps the angle (exploratory). Sign hashes discard how far each projection is from zero. RaBitQ [52] keeps one bit per dimension of a randomly rotated vector, as a hash does, but uses all of them in an unbiased estimate of the inner product with a known error bound. With zig-rabitq [53], written for this check, ranking every chunk by the estimate from 1024-bit codes and a 4-bit query, and rescoring only the top 100 by exact cosine (1.9 percent of the corpus), gave recall at 10 of 0.698, equal to exhaustive cosine (0.698); the top 25 alone gave 0.695. Extended RaBitQ codes [54], with several bits per dimension, shorten the list: at four bits the top 25 estimates contained every exact top-10 chunk (1.000 against 0.978 at one bit). Product quantization [16] at the same 128 bytes per vector, with zig-pq [55] checked against Faiss [17], performed alike (0.978 of the exact top-10 chunks among the top 25, 0.999 among the top 100); at 64 bytes it needed about twice as many candidates, and an inverted file probing 8 of 64 lists capped at 0.898, because the true neighbours of some questions sit in lists it does not visit. The estimate still reads one code per chunk, so at this size it saves arithmetic and memory rather than scans; but as a compact key it does what the Hilbert key was meant to do, cutting the exact rescoring to about two percent of the chunks without losing recall.

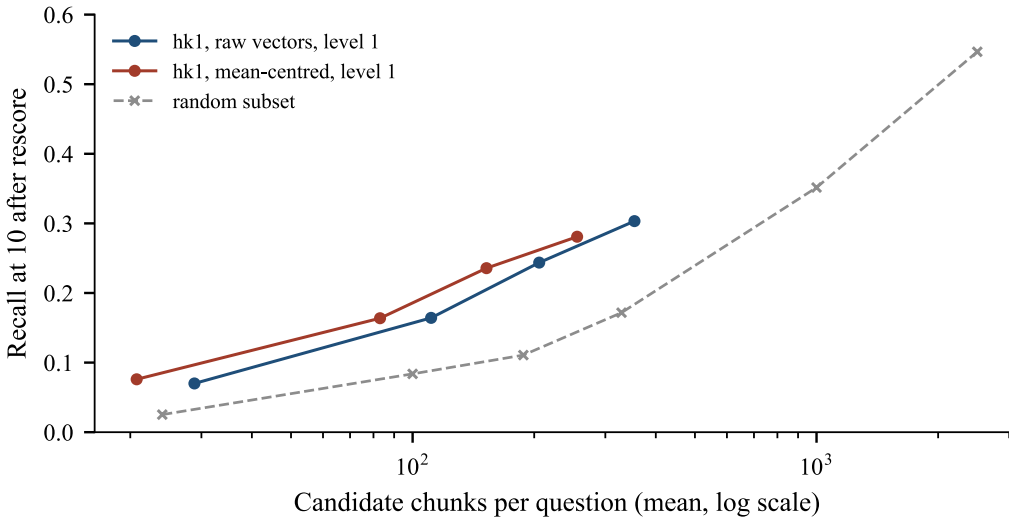


Figure 17: Level-1 filter recall against candidates for raw and mean-centred keys, with the random control.

6.3 Where dense retrieval fails

Dense chunk retrieval placed no relevant note in the top ten for 190 questions (Figure 18). In 7 the relevant note no longer existed in the live corpus, and in 3 a duplicate copy of the relevant note under a second slug was retrieved instead; these are label errors, not retrieval errors. Of the rest, 110 had the relevant note within the top 50, within reach of a reranker or a larger budget, and 70 ranked it below 50, where no reordering of the top of the list helps.

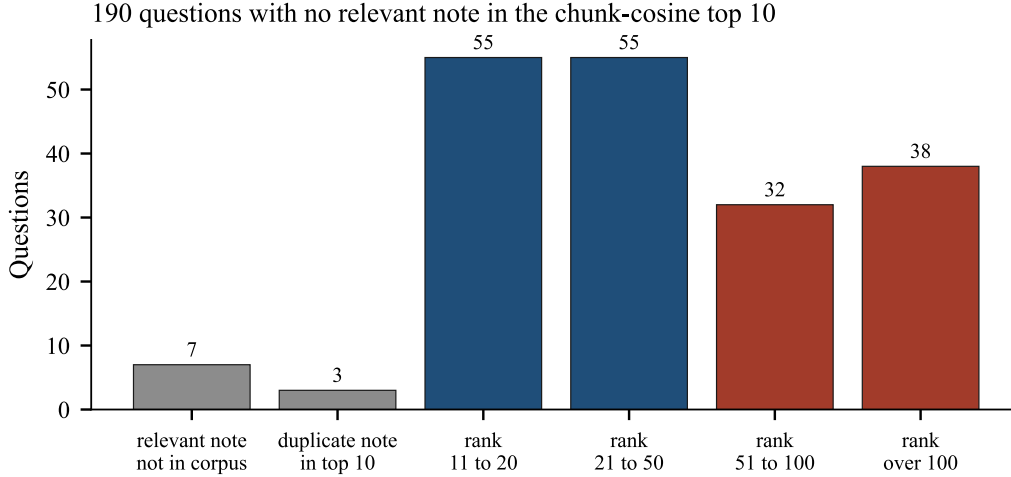


Figure 18: Failure taxonomy of dense chunk retrieval on development questions with no relevant note in the top 10.

7 Exploratory study: twelve ideas from the literature

The confirmatory study left two measured weak points: 190 questions with no relevant note in the dense top ten, of which most have it within the top 50 (Section 6.3), and the evidence pack, which fits only about ten notes into 6,000 tokens (Section 5.3). A survey of the cited preprints and related recent work produced twelve ideas aimed at these points or at the Hilbert key. This section tests them as a separate exploratory family.

7.1 Design

Each idea was implemented on the same 817 development questions; the sealed held-out set was not used. A pre-registration fixing every hypothesis, metric, test, threshold and tunable value was committed before any idea ran (`bench/ideas/preregistration.md`). The development questions were split once into a tuning half and a confirmation half; any choice among variants was made on the tuning half and tested on the confirmation half. The 16 hypotheses of the family were Holm-corrected together, with hypotheses that could not be run entering at $p = 1$. An idea was accepted only if its point estimate met its pre-stated threshold, its family-wide Holm p was below 0.05, and any guard check passed. One deviation was recorded before the affected ideas ran: the non-inferiority tests (ideas 6, 7 and 11) use the larger p of the shifted Wilcoxon test and a paired t -test, for the reason given in Section 4.7.

Table 12: The exploratory family. Effects are paired differences in recall at 10 against dense chunk retrieval unless marked; for ideas 6 and 7 the comparator is exhaustive cosine and the threshold is a non-inferiority margin. Effects for ideas 2, 3, 5, 6, 7, 9 and 10 are on the confirmation half (408 questions), idea 12 on its subset of 314 questions, the rest on all 817. Source: bench/ideas/summary.json.

Id	Idea and source	Effect (95% CI)	Threshold	Holm p	Outcome
H1a	Rerank dense top 50, default instruction [12], [56]	0.036 (0.014 to 0.059)	0.03	0.007	accepted
H1b	Rerank dense top 50, corpus instruction [12]	0.012 (−0.011 to 0.035)	0.03	1.000	not accepted
H2	Sentence-pruned packing of the top 30 (survival) [57]	0.098 (0.071 to 0.127)	0.03	< 0.001	not accepted
H3	Corpus-specific query instruction [12]	0.002 (−0.014 to 0.019)	0.02	1.000	not accepted
H4a	Subtract the shared mean [23], [58]	0.005 (−0.008 to 0.017)	0.02	1.000	not accepted
H4b	Subtract separate query and chunk means [58]	−0.017 (−0.034 to −0.002)	0.02	1.000	not accepted
H4c	Project off the shared mean direction [58]	0.004 (−0.007 to 0.016)	0.02	1.000	not accepted
H4d	Project off separate directions [58]	−0.015 (−0.031 to 0.001)	0.02	1.000	not accepted
H5	Rocchio feedback on vectors [59]	0.002 (−0.013 to 0.017)	0.02	1.000	not accepted
H6	Hilbert partitions with SOAR assignment, vs exhaustive [19], [60]	−0.043 (−0.066 to −0.023)	−0.03	1.000	not accepted
H7	Many randomised Hilbert orders, vs exhaustive [61]	−0.059 (−0.090 to −0.028)	−0.03	1.000	not accepted
H8	Per-chunk synopses from a local model [37]	—	0.03	1.000	not run
H9	Link-graph personalised PageRank [62]	0.011 (0.001 to 0.022)	0.02	0.902	not accepted
H10	HyDE draft answer fused with the question [63], [64]	0.009 (0.001 to 0.020)	0.03	0.414	not accepted
H11	Adaptive cut-off at the largest score gap (survival) [65]	−0.192 (−0.220 to −0.166)	−0.02	1.000	not accepted
H12	MUVERA encodings over hk1 buckets, notes with 5+ chunks [5]	−0.258 (−0.305 to −0.213)	0.03	1.000	not accepted

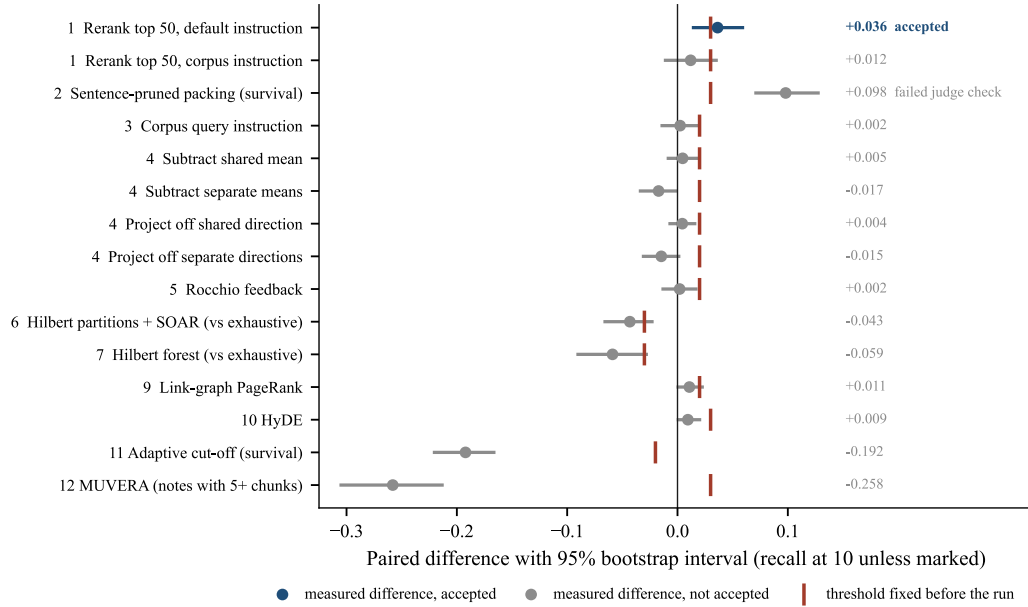


Figure 19: Each idea’s effect against its own pre-stated threshold (red tick). Idea 2 exceeds its threshold but failed its guard check, so it is not accepted.

7.2 Results

Accepted: reranking the dense top 50. Reordering the dense top 50 notes with the cross-encoder reranker and its default instruction raised recall at 10 by 0.036 (95% CI 0.014 to 0.059, family Holm $p = 0.007$). Of the 111 questions whose relevant note sat at ranks 11 to 50, 56 moved into the top ten, while 32 earlier top-ten hits dropped out. The cost is latency: 4.4 s median and 6.2 s at the 95th percentile per question. The corpus-specific instruction did worse than the default (0.012, not significant). Appendix A had found no gain from the same reranker; there it reordered the weaker hybrid list, not the dense one.

Close but not accepted. Sentence-level pruning of the top 30 notes kept the relevant note in the pack far more often than chunk packing (+0.098 survival) with fewer tokens. Survival, however, rises almost by construction when more notes fit, so the pre-registered guard asked a blind judge, the local 27B model, whether the packed text answers the question on 100 questions. It did so for 77 percent of pruned packs against 82 percent of chunk packs, a loss beyond the two-point limit, so the idea is not accepted. Link-graph PageRank passed its gate (96 percent of the deep misses lie within two links of a top-ten note) and gained 0.011 in recall at 10, below its +0.02 threshold. HyDE gained 0.009 and added 14 s per question for the draft answer.

Null and negative results. A corpus-specific query instruction, all four variants of mean-direction removal, and Rocchio feedback changed recall by less than 0.02 in either direction. Hilbert-ordered partitions with a SOAR-style second assignment closed most of the gap left by the hk1 filter (Section 5.4): at 18 percent of chunks scanned they ended −0.043 behind exhaustive cosine, against −0.394 for the hk1 probes, but missed the 0.03 margin, and ordinary k-means partitions at a similar scan (923 candidates) reached 0.682 against 0.699 for exhaustive search. Many randomised Hilbert orders did worse (−0.059). The adaptive cut-off removed most tokens but lost the relevant note for many questions (−0.192 survival). MUVERA encodings over hk1 buckets fell far behind best-chunk scoring (−0.258): with single-vector queries the encoding reduces to scoring one bucket. Per-chunk synopses were not run: one synopsis took 18.2 s on the local model, projecting to 26 h for the corpus, beyond the 10-hour budget. Late chunking [66] was not applicable, because Qwen3-Embedding pools only the last token.

7.3 Implementation plans

Each plan below is stated as work to be done and verified, not as a decision taken.

- **Reranking (accepted).** Feed gbrain’s reranker from the dense chunk ranking at depth 50 with the default instruction. Measure depth 30, a shorter document cap and score caching against the 4.4 s cost, confirm the configuration once on the held-out set, then measure it end to end in the agent harness.
- **Link-graph PageRank (close).** Pre-register a one-hop, hub-down-weighted variant stacked on the reranker, implemented as a gbrain post-processor over the existing link table.
- **Sentence pruning (close).** Pre-register a hybrid pack: full chunks for the top ten, pruned sentences for ranks 11 to 30, with a sentence cap that uses the whole budget (pruned packs used 65 percent of it). Validate the judge against human labels first.
- **Partitioned index (descriptive).** Not needed at five thousand chunks. If the corpus grows by an order of magnitude, test pgvector’s IVF index against k-means partitions before adding a partitions command to zig-hilbert.

8 Exploratory study: relevant notes per token

The evidence pack is what the model reads, so the question here is narrower than retrieval: at the same number of tokens, can a different choice of units put a relevant note into the pack more often? Eight methods from optimisation, diversity, graph diffusion and conformal prediction were tested as a second exploratory family.

8.1 Design

The pre-registration (`bench/efficiency/preregistration.md`) fixed every hypothesis, grid, test and threshold before any data were computed, and the implementation details it left open were committed separately before the first result (`bench/efficiency/deviations.md`). The outcome is survival, whether at least one relevant note is in the pack, at budgets of 1,000, 2,000, 3,000, 4,000 and 6,000 tokens; the primary outcome per question, $\bar{S}$, is its mean over the five budgets. A ratio such as relevant notes per token is not used, because a pack of one chunk would maximise it. The baseline is the chunk pack of [Section 5.3](#): cosine ranking, the top ten notes walked in rank order, each winning chunk kept whole if it fits. Methods that select or reorder units worked on the winning chunks of the top 50 notes; methods that re-rank scored all notes and packed with the baseline rule. Every parameter was chosen on the tuning half of the development questions and tested once on the confirmation half, with the split of [Section 7](#). The primary test is the one-sided Wilcoxon signed-rank test on $\bar{S}$, Holm-corrected over the eight ideas; an idea is accepted if its gain is at least +0.02 and its Holm p is below 0.05.

The methods were implemented in Python as pre-registered. As a check on the implementation, the chosen setting of each of M1 to M7 was recomputed on 20 confirmation questions with two independent libraries written for this study, zig-select [\[67\]](#) and zig-diffuse [\[68\]](#): all selections and orders were equal, and diffused scores agreed to within 2.8×10^{-9} (the conjugate-gradient tolerance of manifold ranking; the other differences were at rounding level).

Table 13: The per-token family, M1 to M7. $\Delta\bar{S}$ is the paired difference in mean survival over the five budgets against the chunk pack, on the confirmation half (408 questions; the baseline’s $\bar{S}$ there is 0.650). None is accepted. M9 is reported in the text. Source: `bench/efficiency/summary.json`.

Id	Method and source	Chosen	$\Delta\bar{S}$ (95% CI)	Holm p
M1	Calibrated relevance per token, greedy fill [69], [70]	pool = 10	−0.054 (−0.070 to −0.038)	1.000
M2	Budgeted submodular coverage [71]	$\lambda = 2$, $p = 0.5$	−0.333 (−0.372 to −0.294)	1.000
M3	Maximal marginal relevance [72]	$\lambda = 0.9$	−0.009 (−0.020 to 0.001)	1.000
M4	Determinantal point process, greedy MAP [73], [74]	$\alpha = 20$	−0.006 (−0.017 to 0.003)	1.000
M5	Heat diffusion on the link graph [75], [76]	$t = 0.5$, $\beta = 0.1$	−0.021 (−0.033 to −0.009)	1.000
M6	Manifold ranking on a neighbour graph [77]	$k = 20$, $a = 0.5$, $\beta = 0.3$	−0.017 (−0.031 to −0.003)	1.000
M7	Advection–diffusion on directed links [78]	$t = 0.5$, $\gamma = 0.25$, $\beta = 0.1$	−0.069 (−0.092 to −0.046)	1.000

8.2 Results

No method is accepted. Every method of M1 to M7 lowered mean survival on the confirmation half. The two diversity methods came closest and are within noise of the baseline: maximal marginal relevance (−0.009) and the determinantal point process (−0.006). Their tuned settings were the grid values that give the least weight to diversity, and the three diffusion methods likewise chose the smallest diffusion they were offered. The closer each method stayed to the plain cosine order, the less it lost. Heat diffusion and manifold ranking left recall at 10 almost unchanged (0.699 and 0.700 against 0.701) but still moved relevant notes out of the packed top ten. Submodular coverage lost most (−0.333): rewarding coverage per token fills the budget with short chunks that resemble many others, and at 6,000 tokens its packs held 0.59 relevant notes per question against 0.92 for the baseline. Ranking by calibrated relevance per token (M1) favours short chunks for the same reason, which costs most at the smallest budget: survival at 1,000 tokens fell from 0.414 to 0.333.

Conformal stopping saves tokens but misses its threshold. M9 keeps the baseline order and stops the 6,000-token pack at the first unit whose cosine is below a threshold calibrated on the tuning half by conformal risk control [79], with a target miss rate of 0.277 (the tuning half’s baseline miss rate plus 0.02). The calibrated threshold was 0.436. On the confirmation half the pack fell from 5,486 to 5,040 tokens per question, 8.1 percent ($p < 0.001$ after Holm), and survival stayed non-inferior at the 0.02 margin (0.760 against 0.767, $p = 0.001$). The realised miss rate, 0.240, stayed below the target, consistent with the method’s guarantee, which bounds the expected miss rate on new questions. The pre-registered criterion was a 20 percent saving, so M9 is not accepted.

What this rules out (efficiency). On this corpus the pack does not lose relevant notes because it repeats itself or because related notes are ranked too low; the plain cosine order is already the best order tested. The remaining losses are questions whose relevant note is ranked below the top ten (Section 6.3), which the reranker of Section 7 addresses. A calibrated stop is the only method that changed cost without changing survival; whether saving 8 percent of the tokens is worth a threshold to maintain is a question of cost, not of retrieval, and would need its own pre-registered test at the budget gbrain uses.

9 Factorial study: every combination of the pipeline

The previous sections changed one switch at a time. This section crosses eight switches of the pipeline in a full factorial, so that each one’s contribution can be read with every combination of the others, and then checks the two best combinations once on the held-out questions.

9.1 Design

The pre-registration (`bench/factorial/preregistration.md`) crosses first-stage search (dense, keyword, or their reciprocal-rank fusion), note scoring, the reranker of [Section 7](#), link PageRank, mean removal, Rocchio feedback, a candidate filter, and the evidence unit, all with the parameters of [Section 7](#) and none re-tuned. The valid combinations are 1,020 cells, 204 rankings times five units, each evaluated on all 817 development questions. A factor’s effect is summarised by its Shapley value: its average contribution to a metric over all combinations of the other factors, with a bootstrap interval over questions. Latency is composed from component timings measured once on this machine. The best configuration on each axis is chosen on the tuning half; the held-out step (deviation D8) runs the best-recall and the best-survival configurations once on the held-out questions, paired with the reference configuration of [Section 5.1](#), dense chunk retrieval with a chunk pack.

The study ran over two nights and recorded eleven deviations (`bench/factorial/deviations.md`). Three matter for reading it. The first night ended in a kernel panic from memory exhaustion, traced to gbrain’s two MLX model servers, which kept up to 24 GB each of freed GPU memory; a 2 GB cache limit fixed it. The second level, an agent harness, ran only its local stratum: the cloud runs were dropped by the author, and runs with a 128K context window did not fit in this machine’s 32 GB. And the level-2 judge ran on a work account by mistake, so its scores are not used here.

Table 14: Shapley values of the factorial’s factors on the 817 development questions. Evidence units do not change the ranking, so their recall value is zero. Source: `bench/factorial/level1-shapley.json`.

Factor (against its reference level)	R@10	Survival	Tokens	Latency (ms)
Keyword search instead of dense	−0.067	−0.101	6	−50
Hybrid of keyword and dense (RRF)	0.010	0.001	77	35
Note mean instead of best chunk	−0.005	−0.007	−10	−0
Reranker on the top 50	0.050	0.057	10	4,364
Link-graph PageRank	0.016	0.014	32	103
Mean-direction removal	−0.002	−0.002	9	0
Rocchio feedback	0.001	0.000	13	0
hk1 candidate filter	−0.367	−0.350	−151	−42
Hilbert partitions filter	−0.050	−0.048	−1	2
Unit: chunk with neighbours	0.000	−0.080	86	0
Unit: section	0.000	−0.021	88	0
Unit: whole note	0.000	−0.130	−114	0
Unit: pruned sentences of the top 30	0.000	0.110	−1,701	1

9.2 Results

What carries recall and survival. The reranker contributes most to recall (0.050) at a cost of 4.4 s per question; link PageRank adds 0.016 for 103 ms, and hybrid search 0.010. Mean removal, Rocchio feedback and note-mean scoring are within a few thousandths of zero. The hk1 filter costs −0.367 in recall in every combination, consistent with [Section 5.4](#). Among evidence units, packing the best sentences of the top 30 notes raises survival by 0.110 with 1,701 fewer tokens, and whole notes lower it by 0.130.

Held-out confirmation. On the 817 held-out questions the reference configuration reached recall at 10 of 0.725, the value of [Section 5.7](#). The best-survival configuration (hybrid + mean-of-chunks + PPR + Rocchio + unit=pruned) kept a relevant note for 0.897 of questions against 0.752, a gain of 0.146 (95% CI 0.121 to 0.171) with 1,517 fewer tokens; 124 questions favoured it and 5 the

reference. The best-recall configuration (hybrid + mean-of-chunks + rerank + PPR), which gained 0.051 on the confirmation half, gained only 0.016 on held-out questions (95% CI -0.009 to 0.041 , Holm $p = 0.49$); its development gain did not replicate at significance. The survival gain of sentence packs replicates.

Do sentence packs answer the question? A separate pre-registered check (`bench/factorial/judge-preregistration.md`) put both packs of 150 questions from the confirmation half to the judge of [Section 7](#), the local 27B model, which was asked whether the pack contains the information needed to answer; the two packs of a question were judged in a random order. The best-survival configuration’s sentence packs were judged sufficient for 0.713 of questions and the reference chunk packs for 0.793, a difference of -0.080 (95% CI -0.147 to -0.013), so non-inferiority at the 0.02 margin is not accepted ($p = 0.96$). Of the questions on which the judge disagreed between packs, 7 favoured the sentence pack and 19 the chunk pack (exact McNemar, two-sided $p = 0.029$). On the same questions the sentence packs kept a relevant note more often (0.893 against 0.753) with 1,443 fewer tokens, but when a relevant note was present they were judged sufficient less often (0.761 against 0.867), which suggests that the five sentences kept per note can leave out what the answer needs; which text was lost was not examined. On this corpus survival is therefore not a stand-in for answer sufficiency, and the survival gain of sentence packs does not carry over to answers.

The local agent. 40 runs of the local 27B model compared 8K and 32K context windows on the pilot tasks of the agent harness. The 32K window used 2.1 times the input tokens and 2.6 times the wall time of the 8K window, and found the target note in 0.35 of runs against 0.40. The sample is too small for a test; the larger window bought no measured gain.

10 Follow-up checks

Six further checks were pre-registered together (`bench/followup/preregistration.md`) after the factorial study, one on the author’s idea for Hilbert keys and five suggested by the literature. Their departures from the pre-registration are recorded in `bench/followup/deviations.md`. A seventh check, on a Hilbert key over attributes the notes already carry, was pre-registered later in the same file.

Hilbert keys as a map for compositional questions. The checks so far used a Hilbert key as a filter for one question vector. The author’s idea is different: an agent splits a question such as “who in a team contributed to open source” into facets, each facet has a region of key cells, and the answer lies in the intersection of the regions. To test it, 60 questions of the form “Which notes are about both A and B?” were drawn from pairs of notes A and B that share 3 to 15 linking notes (2,090 pairs qualified), and the gold answer is the set of notes linking to both. Dense retrieval of the whole question found 0.460 of the gold in its top 10 and 0.704 in its top 30. Intersecting the top 50 notes of each facet, retrieved densely, found 0.479 at a precision of 0.240. Intersecting the facets’ `hk1` regions (level 1, 16 ranges) found 0.182 at a precision of 0.022, a difference of -0.297 from the dense intersection (Holm $p = 7.5 \times 10^{-7}$); each facet’s region held on average only 0.367 of the gold notes, so the intersection loses most of them. The links themselves answer every question exactly, by construction of the gold. On these measurements an embedding-derived Hilbert key does not carry facets well enough to be intersected; a key built over explicit attributes, the classic use of the curve for multi-attribute indexing, is tested below.

The same questions for an agent and on sentences. The local 27B agent of level 2 answered 20 of these questions with three tool sets. With a tool returning the dense facet intersection it found 0.196 of the gold notes using 8,156 input tokens and 64 s on average; with the `hk1` facet tool it found 0.033 using 19,830 tokens and 148 s; with search alone 0.013 using 24,150 tokens (descriptive, no test). Keys on 165,279 sentences instead of chunks raised the `hk1` intersection’s set recall to 0.964, but only because a facet’s region then covers most notes: the median answer held 869 notes at a precision of 0.005. Before this run the author fixed an acceptance rule: a slower key is acceptable if it spends at least 25 percent fewer model tokens per correct answer and at most twice the time. Against dense intersection on the same sentences, sentence keys needed 605 times the evidence tokens per correct answer while taking 0.22 of the time; chunk keys needed 7.5 times the tokens and 3.2 times the time; the agent with the `hk1` tool, judged after the fact, 12.2 and 11.5 times. The rule rejects the key in every comparison. Dense intersection on sentences itself found 0.924 of the gold with a median of

12 notes and 151 evidence tokens per correct answer; because a note that links to A usually names A in some sentence, this figure may reflect how the gold was built. The author checked the first 30 gold lists and marked 25 of them correct and complete. On those 25 questions the hk1 intersection’s set recall is 0.176 against 0.426 for dense intersection (Holm p 0.002), the same ordering as on all 60.

A Hilbert key over explicit attributes. The seventh check gives the key attributes on which nearness is defined and nothing is learned: a note’s area (the first two parts of its path), its type, and the month of its date. Retrieval comes first. Questions of the form “Which notes in area P are about B?” were drawn, where B is a note outside P and the gold is the notes in P that link to B. The caps on how often an area may recur allowed 38 pairs instead of the planned 60 (deviation D13). Taking the top 50 notes of a dense search on B’s title and keeping those in P found 0.484 of the gold at a precision of 0.410. Intersecting the same 50 with a dense search on the area’s words, as the facets were intersected above, found 0.163, a difference of 0.321 (Holm p 2.4×10^{-5}). Against the whole question’s top 30 (0.414) the filter’s difference of 0.070 was not significant (Holm p = 0.15), and its median answer held 11,204 tokens against 224,289. The level-1 hk1 region of B’s title, kept to P, found 0.188 (difference -0.296 , Holm p 8.0×10^{-5}). With the area cap raised until 60 pairs were drawn, an exploratory check, the order was the same: 0.465 for the filter, 0.117 for the area’s words and 0.386 for the top 30. An attribute helps as a filter, not as words to embed, and the embedding key does not help even with the filter.

Then the index. Each note’s area, type and month, as ordinals of 8 bits, were given a three-dimensional Hilbert index, checked to visit every cell of a grid 16 cells on a side once, in unit steps. It was compared with a Z-order key, a composite B-tree on (area, type, month), and three single-column B-trees combined by bitmap AND, each in a table stored in its own index order, on PostgreSQL 18.6. A query is a box: an area and a type, an area alone, or a type alone, over 1 to 24 months, 60 boxes of each shape, at the 1,186 live notes and at 200,000 rows drawn from their attributes. Curve keys cover a box with at most 64 ranges. The composite B-tree touched the fewest buffers for every shape at both sizes (Table 15). At 200,000 rows the Hilbert key touched 1.09 to 1.27 times the composite B-tree’s buffers and took 1.21 to 1.28 times its time, so the pre-registered rule, at least 25 percent fewer buffers in at most twice the time, rejects it for every shape. Against Z-order the Hilbert key needed a median of 114.5 exact ranges for a type over all areas against 216, and touched 8 percent fewer buffers. With an area fixed the two were level (ratio 1.00). The bitmap AND, its rows in random order, touched the most. On these attributes the curve gives nothing that a composite B-tree does not already give.

Table 15: Attribute boxes at 200,000 rows: median shared buffers touched and median execution time over 60 boxes per shape, each box over 1 to 24 months. “Area” fixes one area and any type, “Type” one type and any area. Source: bench/followup/s9b.json.

Access path	Area and type		Area		Type	
	Buffers	ms	Buffers	ms	Buffers	ms
Hilbert key	131	0.27	603	1.12	634	1.19
Z-order key	127	0.27	602	1.12	686	1.29
Composite B-tree	118	0.22	475	0.88	581	0.99
Bitmap AND	3,031	1.75	5,760	2.09	6,107	2.62

Is the judge’s verdict on sentence packs about length? Sentence packs are shorter than chunk packs, and language-model judges are known to favour longer text [80]. For each of the 150 questions of the answer-quality check, the chunk pack was cut to the token count of that question’s sentence pack (3,737 tokens on average) and judged again with the same prompt, which asks whether the context is sufficient to answer [81]. The cut chunk packs were judged sufficient for 0.793 of the questions, the same as the full chunk packs (0.793); against the sentence packs (0.713), 19 questions favoured the cut chunk pack and 7 the sentence pack (exact McNemar p = 0.029). By the pre-registered reading, content and not length explains the gap, which suggests that the five sentences kept per note leave out what whole chunks contain.

The author’s labels on the same questions. He labelled both packs of 50 of the 150 questions, one pack at a time, with the condition and the judge’s answer hidden. The judge agreed with him on 0.830 of the 100 packs (Cohen’s kappa 0.50), 0.800 on sentence packs and 0.860 on chunk packs. His yes-rate was 0.800 for sentence packs and 0.800 for chunk packs, a paired difference of 0.000 (95% CI -0.112 to 0.112). Prediction-powered inference [82] takes the judge’s mean on all 150 questions and adds the mean of (author minus judge) on the 50 labelled questions. For sentence packs that estimate is 0.753 (95% CI 0.609 to 0.898), for chunk packs 0.813 (95% CI 0.690 to 0.937), and for the difference -0.060 (95% CI -0.227 to 0.107). The judge-only difference on all 150 questions is -0.080 (95% CI -0.146 to -0.014). The author’s labels put the two packs level, and the corrected interval for the difference includes zero.

Compression baselines. Three further packs were judged on the same questions. A hybrid that keeps whole chunks for the top ten notes and pruned sentences for ranks 11 to 30 was judged sufficient for 0.807 of questions with 5,972 tokens; LongLLMLingua [83], compressing the chunk pack to the sentence pack’s length, for 0.793 with 3,680 tokens; the extractive compressor of RECOMP [84], choosing sentences up to the same length, for 0.627 with 3,972 tokens; the full chunk pack scored 0.793 with 5,415 tokens. None passed the pre-registered non-inferiority test against the chunk pack at the 0.02 margin (hybrid $p = 0.11$, LongLLMLingua $p = 0.07$), so with 150 questions none can be called as good. Two differences stand out: question-aware token compression matched the chunk pack’s rate, as a point estimate, with 32 percent fewer tokens, and an extractive compressor trained on English questions did worse than the corpus’s own sentence ranking (15 against 2 discordant questions, $p = 0.002$).

Cross-polytope hashing and whitening. Andoni et al. [85] prove that cross-polytope LSH, which hashes a rotated vector to its largest coordinate, needs fewer tables than random hyperplanes at the same recall. A new library [86] implements both families over the same seeded rotations. Hyperplane tables of eight bits reproduced hk2 (R@10 0.607 at 32 tables). The pre-registered comparison at 10 percent of chunks scanned could not be made, because cross-polytope tables scan less: at 128 tables without probes it reads 9.4 percent. At matched cost, an exploratory comparison (deviation D4), cross-polytope reached 0.676 reading 9.4 percent of chunks against 0.607 at 13.0 percent for hyperplanes (difference 0.069, 95% CI 0.045 to 0.094), at four times the rotations per query. Whitening the embeddings [87] lowered recall for both families and for exhaustive cosine (0.643 against 0.698, $p 2.2 \times 10^{-5}$). Removing the mean, as in Section 6.2, is what makes the tables selective: at 32 hyperplane tables, raw vectors scanned 33.6 percent of chunks for R@10 0.669, centred ones 13.0 percent for 0.607.

How many questions the reranker comparison needs. Only 19 percent of the development questions change recall at 10 between the reranked and the reference configuration. Detecting a gain of the held-out size (+0.016) with 80 percent power [88], by a paired bootstrap of the two-sided Wilcoxon test, needs about 1,079 questions; at 800 questions the power is 66 percent, and the held-out set has 817. The development gain (+0.044) needs 387. The held-out result is therefore uninformative rather than negative (deviation D3 explains the method).

How incomplete are the relevance labels? Each development question has the notes its author marked relevant, and a note the retriever found but nobody marked counts as a miss [89]. For 80 development questions (100 were drawn; the pre-registered cap on judge calls reduced them), the notes in the top ten of the reference configuration or of the best-recall configuration that were not labelled relevant, 1,059 pairs, were shown to the local judge with a relevance prompt after Thomas et al. [90]. It called 42 percent of them relevant (95% CI 36 to 49), and 76 of the 80 questions had at least one such note. Adding them to the labels lowered the reference configuration’s recall at 10 from 0.761 to 0.727, because its denominator grew, and raised the reranked configuration’s from 0.788 to 0.847: the reranker’s gain on these questions went from 0.026 ($p = 0.25$) to 0.119 (95% CI 0.083 to 0.157, $p < 0.001$). By the judge’s labels, the reranker surfaces relevant notes that the original labels miss. Two cautions apply. The judge and the reranker are models of the same family and may share preferences. On 30 pairs the author labelled afterwards, 15 the judge called relevant and 15 it did not, he agreed with 0.733 of the relevant calls and with 0.533 of the others (overall agreement 0.633, kappa 0.27).

11 Specified cells on the same switches

The literature scan of 8 October 2026 proposed further changes to this corpus. Several of them are already closed here. Late chunking [66] does not apply, because Qwen3-Embedding pools only the last token. HyDE [63], link-graph PageRank, sentence pruning [57], and a pack cut off at the largest cosine gap [65] have been run, and their results are in Section 7, Section 8 and Section 9. The note-mean condition of Section 5.2 already scores each note by the mean of its current chunk vectors, so there is no separate stored mean to refresh.

Six cells on switches this paper already measures have been run on the development questions. The protocol is one factor at a time, with the reranker off except where it is the factor. The held-out questions stay sealed, because no cell raised recall at 10 on the development set.

Ranking by the best sentence. RQ2 compared the note mean with the best chunk. The sentence packs of Section 9 kept sentences of notes that chunk ranking had already selected, and those packs were judged sufficient less often than chunk packs. Here a note’s score is the maximum cosine of its stored sentence vectors with the question. Recall at 10 is 0.660 against 0.698 for best-chunk ranking. The difference is -0.039 (95% CI -0.067 to -0.011). The interval sits below zero. Source: Chen et al. [91].

Seed-clamped smoothing. Link PageRank in Section 7 gained recall below its threshold, and diffusion in Section 8 moved relevant notes out of the packed top ten. From seed scores s , the cell iterates $p \leftarrow \alpha s + (1 - \alpha)Wp$ on the undirected link graph and ranks by $\max(p, s)$. Recall at 10 is 0.698 against 0.698. The difference is -0.001 (95% CI -0.006 to 0.005), and the original top note stayed inside the top ten for all 817 questions. Source: Miao et al. [92].

Local forward push. The PageRank factor in Section 9 added 103 ms. A forward push from the same seeds, on the directed link graph, has recall at 10 of 0.720 against 0.713 for the existing power iteration. The difference is 0.007 (95% CI -0.001 to 0.014). The push took 0.000288 s per question and the power iteration 0.000194 s. Source: Wang et al. [93].

The reranker only when the top two scores are close. The reranker raises recall and costs seconds (Section 7). The threshold is the tuning-half gap at which the reranker is called, chosen as the smallest call rate whose MRR interval includes zero, then applied once on the confirmation half. The chosen threshold is 0, so the reranker runs only on exact ties, on 0.005 of the confirmation questions. Confirmation MRR is 0.528 against 0.545 for always calling it. The difference is -0.017 (95% CI -0.055 to 0.020). Time on that half is 0.16 s against 4.56 s. Source: Zhao et al. [94].

Stopping when the packed text can answer. Conformal stopping on cosine (Section 8) saved 8.1 percent of tokens and missed a pre-registered 20 percent target, while leaving survival within 0.02. Here chunks of the reference pack are added, in best-chunk order, until JEV-9B assigns probability at least one half to the pack being able to answer. On the same 150 questions, the stopped packs use 1253 tokens against 5415 for the full chunk pack, which is 77 percent fewer. The 27B judge calls the stopped packs sufficient for 0.687 of the questions, against 0.793 for the full chunk pack. The difference of -0.107 is outside 0.02. JEV-9B reads at most 1,024 tokens of a pack, its head and its tail, so it judged longer packs on a shortened text. It stopped after the first chunk on 110 questions. On 12 it never reached one half, and those packs kept every chunk that fit in order. Source: Jeong et al. [95].

The winning sentence with its headings. Sentence packs reached the note and were judged sufficient less often than chunk packs. The section pack of RQ3 returns text through the next heading. This pack is the winning sentence of each top note, in best-chunk order, together with the markdown headings open above that sentence. No new embedding was made. The packs use 466 tokens against 5415. The same judge calls them sufficient for 0.347 of the 150 questions, against 0.793 for the chunk pack. Source: Rainey et al. [96].

The same scan ran the remaining cells on the development questions, and several lines stopped at a count. Subtracting each note’s mean cosine to the tuning-half queries lowered recall at 10 from 0.698 to 0.628 (difference -0.070 , 95% CI -0.091 to -0.049). Of 186 facts, 3 are marked superseded and 0 notes carry one, so the replacement line stops, and with it the cell that would hide a replaced fact. Pages store no mean of their chunk vectors. 0 live facts share an entity and a kind with two values, so

the disagreement line stops. 15 facts carry an end date and 1 of them cites a note, which is too small a set for a sufficiency or recall comparison, so that cell is the count. Because the early stop already cut tokens by more than 20 percent, the cell that would drop non-evidence stays closed. On the 60 two-page questions, the union of the top ten notes for each facet title, with the two target pages removed and with no link join, has set recall 0.487, above the bar of 0.479. A spread threshold fixed on the tuning half marks 0.293 of confirmation misses and 0.294 of confirmation hits (difference -0.001 , 95% CI -0.103 to 0.108).

12 Discussion

Principal findings. On a personal corpus of about five thousand chunks, two switches account for the measurable gains: dense scoring, and returning chunks rather than whole notes to the model. The first raises recall by about half of the question set; the second raises the share of questions whose relevant note survives a 6,000-token budget by fourteen points at equal token cost, and no question in the set was hurt by it. The note-versus-chunk choice for ranking, the hybrid, the index structure, and the database engine did not change recall.

Relation to prior work. The weakness of lexical retrieval here is less a property of BM25-style scoring [9] than of conjunctive query parsing applied to long natural-language questions; with disjunctive parsing the lexical arm becomes competitive at chunk level, yet still adds nothing to fusion. The equality of note-mean and best-chunk ranking contrasts with the gains reported for set-based scoring on public benchmarks [4], [5]; the notes in this corpus are short (median 2 chunks), so a mean of two vectors loses little. The behaviour of the Hilbert key follows from angular hashing theory [20], [21] once the anisotropy of contextual embeddings [22] is removed [23], and the recall measured while the key was being built, on near-duplicate pairs at cosine 0.96 [8], is consistent with Equation 1: at that cosine the model gives a level-1 collision probability above one half.

Implications. The results suggest that, for a small personal knowledge base, what is returned to the model matters more than how it is indexed. Exhaustive cosine over every chunk costs well under a millisecond per question here and remains sub-millisecond at forty times the size with HNSW. On these measurements a Hilbert key does not work as a retrieval index; making it work as one would take tens of independent keys, which is a locality-sensitive-hashing index rather than a single column. As an index over attributes the notes already carry, area, type and month, it touched more buffers than a composite B-tree for every box shape (Section 10). Its value as a label for browsing notes was not measured. Six further cells on these same switches are reported in Section 11. Calling the reranker only on exact ties met its time bar. The other five did not meet theirs, and the held-out questions stay sealed.

Auxiliary observation: the harness. Two harness factors were measured outside the hypothesis framework (Appendix B). A shell-output filter, RTK [97], halved the bytes of five fixed commands but lengthened one. On thirty questions, gbrain search found the relevant note in 21, exact text search with ripgrep [98] in 0, and word search in 16. The comparison of agent hosts with these tools switched on and off was not run and is left to future work.

13 Threats to validity

The threats are grouped by the four validity types of Wohlin et al. [99].

Construct validity. Relevance labels were generated against the same brain and are binary; 16 relevant slugs no longer exist and a few duplicated notes split relevance. The labels are also incomplete: a local judge called 42 percent of the unlabelled notes in two configurations' top ten relevant (Section 10), so absolute recall values are lower bounds and comparisons that favour a method surfacing unlabelled notes, such as the reranker, are biased against it. Token counts use gbrain's character heuristic, not a model tokenizer.

Internal validity. The live corpus grew from the planned 5,143 to 5,155 chunks before the primary cells ran, and all primary conditions ran within about one hour against the same tables. The hypothesis tests and mechanism analyses reloaded the live corpus later, when it held 5,157 chunks; the chunks-per-note and tokens-per-chunk distributions of Table 2 and Figure 1 come from that load. On the four primary cells the two loads gave the same recall, MRR, and nDCG to six decimals. The

hk1 range at 200,000 rows was a sequential scan. The production-hybrid held-out figure comes from a different evaluator run. The engine comparison measures client-observed latency, which includes HTTP and JSON for ArangoDB and the Postgres wire protocol for pgvector; the IVF and HNSW parameters were swept over a few values, not tuned exhaustively. In the first pgvector runs the planner chose a sequential scan and the client reused cached plans; both were corrected before the reported run.

External validity. One corpus, one author, one embedding model, one language. Corpora with longer notes may favour set-based scoring; corpora with more exact identifiers may favour lexical retrieval. The 200,000-vector set was synthetic, so only its latency, not its recall, is interpreted. The engine comparison covers four alternatives to Postgres, measured at this corpus size only; the embedded engines were timed in process, without the network round trip a server pays.

Software versions. The measurements in the body ran on PostgreSQL 16.15 with pgvector 0.8.6 and ArangoDB 3.12.10. A repeat on PostgreSQL 18.6, pgvector 0.8.7 and ArangoDB 3.12.12 gave the same retrieval quality and the same ordering of the engines ([Appendix C](#)).

Statistical conclusion validity. Per-question outcomes are not independent where questions share relevant notes; the bootstrap resamples questions, not notes. Exploratory analyses, including the centring intervention, were chosen after seeing the confirmatory results and are reported as such.

14 Conclusion

Of the configurations measured, dense chunk retrieval with chunk-level evidence performed best: it found more relevant notes than lexical search (H1), did not differ from note-level dense ranking (H2), and under a fixed budget kept the relevant note far more often than whole notes (H3). A Hilbert-curve key failed as a candidate filter (H4); its first level is an eight-bit sign hash whose collision rate, after mean-centring, matches angular hashing theory, and that theory explains the failure: a question and its answer are too far apart in angle for one key to bring them together. Postgres with pgvector met the latency needs of this corpus at its size and at forty times it, and ArangoDB, measured on the same vectors and links, was slower at both vector search and one-hop expansion without retrieving anything different.

The two exploratory studies sharpen where the remaining losses are. Diversity, coverage and diffusion methods did not put relevant notes into the budget more often than the plain cosine order ([Section 8](#)), so redundancy is not what loses them. Packing the best sentences of more notes did, and that gain held on held-out questions ([Section 9](#)), but a pre-registered judge found those shorter packs sufficient to answer less often than chunk packs, so the gain lies in reaching the note, not in answering; a pack cut to the same length but kept as whole chunks did not lose ([Section 10](#)), which points to what sentence pruning leaves out. The author’s labels on 50 of these questions give both packs the same yes-rate, and the prediction-powered interval for their difference includes zero ([Section 10](#)). A hybrid of whole chunks and sentences, and question-aware token compression with 32 percent fewer tokens, came close to chunk packs without passing a non-inferiority test on 150 questions.

The same switches were measured again after the literature scan of 8 October 2026 ([Section 11](#)). Best-sentence ranking reached recall at 10 of 0.660 against 0.698. Seed-clamped smoothing and the forward push stayed inside the interval of their baselines. The reranker, called only when the top two cosine scores tie, took 0.16 s on the confirmation half against 4.56 s, with MRR inside the always-on interval. A checker that stops the chunk pack early left 77 percent fewer tokens, and the 27B judge called those packs sufficient for 0.687 of the 150 questions against 0.793. The winning sentence with its headings used 466 tokens and was sufficient for 0.347. The held-out questions stay sealed.

The follow-up checks answer two open questions. The reranker’s shrunken held-out gain came from a measurement unlikely to detect it reliably: at the held-out set’s size the power to find a gain that size is about 66 percent, and incomplete relevance labels count many of the reranker’s extra notes as misses. And the Hilbert key, tried as a map whose facet regions an agent intersects, again lost to dense retrieval and failed the author’s own cost rule at chunk and at sentence level; dense intersection on sentences found 0.924 of the link-derived answers with 151 evidence tokens per correct answer, the author marked 25 of the 30 gold lists correct and complete, and on that subset the chunk-level

hk1 intersection still trails dense intersection (Section 10). Built over attributes the notes already carry, the key lost again. The area worked as a filter on dense retrieval, finding 0.484 of the gold against 0.163 when the area was searched as words. As an index, the curve touched more buffers than a composite B-tree for every box shape (Section 10).

Data and code availability

All result files, scripts, and figures are in the repository github.com/guanchzhou/hilbert-paper. Every number in this article is read by the typesetter from a JSON file in `bench/`; `python3 build.py` redraws all figures, records file digests, typesets with Typst [100], and refuses to finish if a figure or result file is missing. The corpus is the author’s private notes and is not published, and neither are note names or question texts; the result files hold counts and scores only.

Competing interests

The author has no financial or commercial interest in any tool evaluated here. zig-hilbert was written by the author as part of this investigation, as the experimental instrument for testing whether Hilbert-curve keys are a useful addition to knowledge management for AI systems; it is not a product being promoted. zig-select and zig-diffuse [67], [68] were written in the same way, as independent checks of the per-token study, and zig-rabitq [53] for the filter comparison. The author uses gbrain as his personal knowledge base. The acceptance criterion for the key was fixed before the sweep.

Funding

None.

References

- [1] N. F. Liu *et al.*, “Lost in the Middle: How Language Models Use Long Contexts,” *Transactions of the Association for Computational Linguistics*, vol. 12, pp. 157–173, 2024, [Online]. Available: <https://arxiv.org/abs/2307.03172>
- [2] S. Robertson and H. Zaragoza, “The Probabilistic Relevance Framework: BM25 and Beyond,” *Foundations and Trends in Information Retrieval*, vol. 3, no. 4, pp. 333–389, 2009.
- [3] V. Karpukhin *et al.*, “Dense Passage Retrieval for Open-Domain Question Answering,” in *Proceedings of EMNLP 2020*, 2020. [Online]. Available: <https://arxiv.org/abs/2004.04906>
- [4] O. Khattab and M. Zaharia, “ColBERT: Efficient and Effective Passage Search via Contextualized Late Interaction over BERT,” in *Proceedings of SIGIR 2020*, 2020. [Online]. Available: <https://arxiv.org/abs/2004.12832>
- [5] R. Jayaram, L. Dhulipala, M. Hadian, J. Lee, and V. Mirrokni, “MUVIRA: Multi-Vector Retrieval via Fixed Dimensional Encodings,” in *Advances in Neural Information Processing Systems 37*, 2024. [Online]. Available: <https://arxiv.org/abs/2405.19504>
- [6] D. Hilbert, “Über die stetige Abbildung einer Linie auf ein Flächenstück,” *Mathematische Annalen*, vol. 38, pp. 459–460, 1891.
- [7] B. Moon, H. V. Jagadish, C. Faloutsos, and J. H. Saltz, “Analysis of the Clustering Properties of the Hilbert Space-Filling Curve,” *IEEE Transactions on Knowledge and Data Engineering*, vol. 13, no. 1, pp. 124–141, 2001.
- [8] A. Maltsev, “zig-hilbert: Hilbert curves, S2 cells, and hk1 knowledge markers in Zig.” [Online]. Available: <https://github.com/guanchzhou/zig-hilbert>
- [9] N. Thakur, N. Reimers, A. Rücklé, A. Srivastava, and I. Gurevych, “BEIR: A Heterogeneous Benchmark for Zero-shot Evaluation of Information Retrieval Models,” in *NeurIPS 2021 Datasets and Benchmarks Track*, 2021. [Online]. Available: <https://arxiv.org/abs/2104.08663>
- [10] G. V. Cormack, C. L. A. Clarke, and S. Büttcher, “Reciprocal Rank Fusion Outperforms Condorcet and Individual Rank Learning Methods,” in *Proceedings of SIGIR 2009*, 2009, pp. 758–759.
- [11] K. Sawarkar, A. Mangal, and S. R. Solanki, “Blended RAG: Improving RAG Accuracy with Semantic Search and Hybrid Query-Based Retrievers.” [Online]. Available: <https://arxiv.org/abs/2404.07220>
- [12] Y. Zhang *et al.*, “Qwen3 Embedding: Advancing Text Embedding and Reranking Through Foundation Models.” [Online]. Available: <https://arxiv.org/abs/2506.05176>

- [13] Y. A. Malkov and D. A. Yashunin, “Efficient and Robust Approximate Nearest Neighbor Search Using Hierarchical Navigable Small World Graphs,” *IEEE Transactions on Pattern Analysis and Machine Intelligence*, vol. 42, no. 4, pp. 824–836, 2020, [Online]. Available: <https://arxiv.org/abs/1603.09320>
- [14] A. Kane and pgvector contributors, “pgvector: Open-source vector similarity search for Postgres.” [Online]. Available: <https://github.com/pgvector/pgvector>
- [15] J. Sivic and A. Zisserman, “Video Google: A Text Retrieval Approach to Object Matching in Videos,” in *Proceedings of the Ninth IEEE International Conference on Computer Vision*, 2003, pp. 1470–1477. doi: [10.1109/ICCV.2003.1238663](https://doi.org/10.1109/ICCV.2003.1238663).
- [16] H. Jégou, M. Douze, and C. Schmid, “Product Quantization for Nearest Neighbor Search,” *IEEE Transactions on Pattern Analysis and Machine Intelligence*, vol. 33, no. 1, pp. 117–128, 2011, doi: [10.1109/TPAMI.2010.57](https://doi.org/10.1109/TPAMI.2010.57).
- [17] M. Douze *et al.*, “The Faiss Library.” [Online]. Available: <https://arxiv.org/abs/2401.08281>
- [18] R. Guo *et al.*, “Accelerating Large-Scale Inference with Anisotropic Vector Quantization,” in *Proceedings of ICML 2020*, 2020. [Online]. Available: <https://arxiv.org/abs/1908.10396>
- [19] P. Sun, D. Simcha, D. Dopson, R. Guo, and S. Kumar, “SOAR: Improved Indexing for Approximate Nearest Neighbor Search,” in *Advances in Neural Information Processing Systems 36*, 2023. [Online]. Available: <https://arxiv.org/abs/2404.00774>
- [20] M. S. Charikar, “Similarity Estimation Techniques from Rounding Algorithms,” in *Proceedings of the 34th Annual ACM Symposium on Theory of Computing*, 2002, pp. 380–388.
- [21] P. Indyk and R. Motwani, “Approximate Nearest Neighbors: Towards Removing the Curse of Dimensionality,” in *Proceedings of the 30th Annual ACM Symposium on Theory of Computing*, 1998, pp. 604–613.
- [22] K. Ethayarajh, “How Contextual are Contextualized Word Representations? Comparing the Geometry of BERT, ELMo, and GPT-2 Embeddings,” in *Proceedings of EMNLP-IJCNLP 2019*, 2019, pp. 55–65. [Online]. Available: <https://arxiv.org/abs/1909.00512>
- [23] J. Mu and P. Viswanath, “All-but-the-Top: Simple and Effective Postprocessing for Word Representations,” in *International Conference on Learning Representations*, 2018. [Online]. Available: <https://arxiv.org/abs/1702.01417>
- [24] K. Järvelin and J. Kekäläinen, “Cumulated Gain-Based Evaluation of IR Techniques,” *ACM Transactions on Information Systems*, vol. 20, no. 4, pp. 422–446, 2002.
- [25] E. M. Voorhees, “The TREC-8 Question Answering Track Report,” in *Proceedings of the Eighth Text Retrieval Conference (TREC-8)*, in NIST Special Publication 500-246. 1999, pp. 77–82. [Online]. Available: https://trec.nist.gov/pubs/trec8/papers/qa_report.pdf
- [26] M. D. Smucker, J. Allan, and B. Carterette, “A Comparison of Statistical Significance Tests for Information Retrieval Evaluation,” in *Proceedings of CIKM 2007*, 2007, pp. 623–632.
- [27] F. Wilcoxon, “Individual Comparisons by Ranking Methods,” *Biometrics Bulletin*, vol. 1, no. 6, pp. 80–83, 1945.
- [28] Q. McNemar, “Note on the Sampling Error of the Difference Between Correlated Proportions or Percentages,” *Psychometrika*, vol. 12, no. 2, pp. 153–157, 1947.
- [29] B. Efron, “Bootstrap Methods: Another Look at the Jackknife,” *The Annals of Statistics*, vol. 7, no. 1, pp. 1–26, 1979.
- [30] S. Holm, “A Simple Sequentially Rejective Multiple Test Procedure,” *Scandinavian Journal of Statistics*, vol. 6, no. 2, pp. 65–70, 1979.
- [31] E. Walker and A. S. Nowacki, “Understanding Equivalence and Noninferiority Testing,” *Journal of General Internal Medicine*, vol. 26, no. 2, pp. 192–196, 2011, doi: [10.1007/s11606-010-1513-8](https://doi.org/10.1007/s11606-010-1513-8).
- [32] G. Tan and gbrain contributors, “gbrain: a personal knowledge brain on Postgres.” [Online]. Available: <https://github.com/garrytan/gbrain>
- [33] Obsidian, “Obsidian: a markdown knowledge base application.” [Online]. Available: <https://obsidian.md/>
- [34] The PostgreSQL Global Development Group, “PostgreSQL 16 Documentation.” [Online]. Available: <https://www.postgresql.org/docs/16/textsearch.html>
- [35] Ollama contributors, “Ollama: run large language models locally.” [Online]. Available: <https://github.com/ollama/ollama>

- [36] A. Hannun, J. Digani, A. Katharopoulos, and R. Collobert, “MLX: Efficient and Flexible Machine Learning on Apple Silicon.” [Online]. Available: <https://github.com/ml-explore/mlx>
- [37] Anthropic, “Introducing Contextual Retrieval.” [Online]. Available: <https://www.anthropic.com/engineering/contextual-retrieval>
- [38] C. L. A. Clarke, G. V. Cormack, and E. A. Tudhope, “Relevance Ranking for One to Three Term Queries,” *Information Processing and Management*, vol. 36, no. 2, pp. 291–311, 2000, doi: [10.1016/S0306-4573\(99\)00017-5](https://doi.org/10.1016/S0306-4573(99)00017-5).
- [39] G. L. Steele, D. Lea, and C. H. Flood, “Fast Splittable Pseudorandom Number Generators,” in *Proceedings of the 2014 ACM International Conference on Object Oriented Programming Systems Languages and Applications (OOPSLA)*, 2014, pp. 453–472. doi: [10.1145/2660193.2660195](https://doi.org/10.1145/2660193.2660195).
- [40] G. Camilli, “Origin of the Scaling Constant $d = 1.7$ in Item Response Theory,” *Journal of Educational and Behavioral Statistics*, vol. 19, no. 3, pp. 293–295, 1994, doi: [10.3102/10769986019003293](https://doi.org/10.3102/10769986019003293).
- [41] S. R. Bowling, M. T. Khasawneh, S. Kaewkuekool, and B. R. Cho, “A Logistic Approximation to the Cumulative Normal Distribution,” *Journal of Industrial Engineering and Management*, vol. 2, no. 1, pp. 114–127, 2009, doi: [10.3926/jiem.2009.v2n1.p114-127](https://doi.org/10.3926/jiem.2009.v2n1.p114-127).
- [42] J. Skilling, “Programming the Hilbert Curve,” *AIP Conference Proceedings*, vol. 707, pp. 381–387, 2004, doi: [10.1063/1.1751381](https://doi.org/10.1063/1.1751381).
- [43] D. S. Kerby, “The Simple Difference Formula: An Approach to Teaching Nonparametric Correlation,” *Comprehensive Psychology*, vol. 3, p. 11.IT.3.1, 2014, doi: [10.2466/11.IT.3.1](https://doi.org/10.2466/11.IT.3.1).
- [44] P. Virtanen *et al.*, “SciPy 1.0: Fundamental Algorithms for Scientific Computing in Python,” *Nature Methods*, vol. 17, no. 3, pp. 261–272, 2020, doi: [10.1038/s41592-019-0686-2](https://doi.org/10.1038/s41592-019-0686-2).
- [45] C. R. Harris *et al.*, “Array Programming with NumPy,” *Nature*, vol. 585, no. 7825, pp. 357–362, 2020, doi: [10.1038/s41586-020-2649-2](https://doi.org/10.1038/s41586-020-2649-2).
- [46] ArangoDB, “ArangoDB 3.12 documentation: vector indexes.” [Online]. Available: <https://docs.arango.ai/arangodb/3.12/indexes-and-search/indexing/working-with-indexes/vector-indexes/>
- [47] A. Garcia, “sqlite-vec: A vector search SQLite extension.” [Online]. Available: <https://github.com/asg017/sqlite-vec>
- [48] Turso, “libSQL: an open-source fork of SQLite with native vector search.” [Online]. Available: <https://github.com/tursodatabase/libsql>
- [49] MongoDB, “MongoDB Search and MongoDB Vector Search on self-managed deployments.” [Online]. Available: <https://www.mongodb.com/docs/search/self-managed/current/>
- [50] P. Jaccard, “The Distribution of the Flora in the Alpine Zone,” *New Phytologist*, vol. 11, no. 2, pp. 37–50, 1912, doi: [10.1111/j.1469-8137.1912.tb05611.x](https://doi.org/10.1111/j.1469-8137.1912.tb05611.x).
- [51] Q. Lv, W. Josephson, Z. Wang, M. Charikar, and K. Li, “Multi-Probe LSH: Efficient Indexing for High-Dimensional Similarity Search,” in *Proceedings of VLDB 2007*, 2007, pp. 950–961.
- [52] J. Gao and C. Long, “RaBitQ: Quantizing High-Dimensional Vectors with a Theoretical Error Bound for Approximate Nearest Neighbor Search,” in *Proceedings of SIGMOD 2024*, 2024. [Online]. Available: <https://arxiv.org/abs/2405.12497>
- [53] A. Maltsev, “zig-rabbitq: RaBitQ binary codes in Zig.” [Online]. Available: <https://github.com/guanchzhou/zig-rabbitq>
- [54] J. Gao, Y. Gou, Y. Xu, Y. Yang, C. Long, and R. C.-W. Wong, “Practical and Asymptotically Optimal Quantization of High-Dimensional Vectors in Euclidean Space for Approximate Nearest Neighbor Search,” in *Proceedings of SIGMOD 2025*, 2025. [Online]. Available: <https://arxiv.org/abs/2409.09913>
- [55] A. Maltsev, “zig-pq: product quantization and IVF-PQ in Zig.” [Online]. Available: <https://github.com/guanchzhou/zig-pq>
- [56] R. Nogueira and K. Cho, “Passage Re-ranking with BERT.” [Online]. Available: <https://arxiv.org/abs/1901.04085>
- [57] N. Chirkova, T. Formal, V. Nikoulina, and S. Clinchant, “Provence: Efficient and Robust Context Pruning for Retrieval-Augmented Generation.” [Online]. Available: <https://arxiv.org/abs/2501.16214>
- [58] X. Ren, Y. Sun, and H. Liang, “Correcting Mean Bias in Text Embeddings: A Refined Renormalization with Training-Free Improvements on MMTEB.” [Online]. Available: <https://arxiv.org/abs/2511.11041>
- [59] H. Li, A. Mourad, S. Zhuang, B. Koopman, and G. Zuccon, “Pseudo Relevance Feedback with Deep Language Models and Dense Retrievers: Successes and Pitfalls.” [Online]. Available: <https://arxiv.org/abs/2108.11044>

- [60] M. AL-Qurishi, Z. Qian, F. AL-Tam, and R. Souissi, “Hildoc: Leveraging Hilbert Curve Representation for Accurate and Efficient Document Retrieval,” in *Proceedings of the 14th International Joint Conference on Natural Language Processing and the 4th Conference of the Asia-Pacific Chapter of the Association for Computational Linguistics*, 2025. [Online]. Available: <https://aclanthology.org/2025.ijcnlp-long.101>
- [61] Y. Imamura, T. Shinohara, N. Higuchi, K. Hirata, and T. Kuboyama, “Hilbert Forest in the SISAP 2025 Indexing Challenge.” [Online]. Available: <https://arxiv.org/abs/2508.21682>
- [62] B. J. Gutiérrez, Y. Shu, Y. Gu, M. Yasunaga, and Y. Su, “HippoRAG: Neurobiologically Inspired Long-Term Memory for Large Language Models.” [Online]. Available: <https://arxiv.org/abs/2405.14831>
- [63] L. Gao, X. Ma, J. Lin, and J. Callan, “Precise Zero-Shot Dense Retrieval without Relevance Labels.” [Online]. Available: <https://arxiv.org/abs/2212.10496>
- [64] L. Wang, N. Yang, and F. Wei, “Query2doc: Query Expansion with Large Language Models.” [Online]. Available: <https://arxiv.org/abs/2303.07678>
- [65] C. Taguchi, S. Maekawa, and N. Bhutani, “Efficient Context Selection for Long-Context QA: No Tuning, No Iteration, Just Adaptive-k.” [Online]. Available: <https://arxiv.org/abs/2506.08479>
- [66] M. Günther, I. Mohr, D. J. Williams, B. Wang, and H. Xiao, “Late Chunking: Contextual Chunk Embeddings Using Long-Context Embedding Models.” [Online]. Available: <https://arxiv.org/abs/2409.04701>
- [67] A. Maltsev, “zig-select: budgeted evidence selection in Zig.” [Online]. Available: <https://github.com/guanchzhou/zig-select>
- [68] A. Maltsev, “zig-diffuse: re-ranking by diffusion on sparse graphs in Zig.” [Online]. Available: <https://github.com/guanchzhou/zig-diffuse>
- [69] G. B. Dantzig, “Discrete-Variable Extremum Problems,” *Operations Research*, vol. 5, no. 2, pp. 266–288, 1957, doi: [10.1287/opre.5.2.266](https://doi.org/10.1287/opre.5.2.266).
- [70] B. Zadrozny and C. Elkan, “Transforming Classifier Scores into Accurate Multiclass Probability Estimates,” in *Proceedings of KDD 2002*, 2002, pp. 694–699.
- [71] H. Lin and J. Bilmes, “A Class of Submodular Functions for Document Summarization,” in *Proceedings of ACL-HLT 2011*, 2011, pp. 510–520.
- [72] J. Carbonell and J. Goldstein, “The Use of MMR, Diversity-Based Reranking for Reordering Documents and Producing Summaries,” in *Proceedings of SIGIR 1998*, 1998, pp. 335–336.
- [73] A. Kulesza and B. Taskar, “Determinantal Point Processes for Machine Learning,” *Foundations and Trends in Machine Learning*, vol. 5, no. 2–3, pp. 123–286, 2012.
- [74] L. Chen, G. Zhang, and H. Zhou, “Fast Greedy MAP Inference for Determinantal Point Process to Improve Recommendation Diversity,” in *Advances in Neural Information Processing Systems 31 (NeurIPS 2018)*, 2018, pp. 5627–5638.
- [75] R. I. Kondor and J. D. Lafferty, “Diffusion Kernels on Graphs and Other Discrete Input Spaces,” in *Proceedings of ICML 2002*, 2002, pp. 315–322.
- [76] F. Chung, “The Heat Kernel as the PageRank of a Graph,” *Proceedings of the National Academy of Sciences*, vol. 104, no. 50, pp. 19735–19740, 2007.
- [77] D. Zhou, J. Weston, A. Gretton, O. Bousquet, and B. Schölkopf, “Ranking on Data Manifolds,” in *Advances in Neural Information Processing Systems 16*, 2004, pp. 169–176.
- [78] A. Chapman and M. Mesbahi, “Advection on Graphs,” in *Proceedings of the 50th IEEE Conference on Decision and Control and European Control Conference*, 2011, pp. 1461–1466.
- [79] A. N. Angelopoulos, S. Bates, A. Fisch, L. Lei, and T. Schuster, “Conformal Risk Control,” in *International Conference on Learning Representations (ICLR 2024)*, 2024. [Online]. Available: <https://arxiv.org/abs/2208.02814>
- [80] L. Zheng *et al.*, “Judging LLM-as-a-Judge with MT-Bench and Chatbot Arena.” [Online]. Available: <https://arxiv.org/abs/2306.05685>
- [81] H. Joren, J. Zhang, C.-S. Ferng, D.-C. Juan, A. Taly, and C. Rashtchian, “Sufficient Context: A New Lens on Retrieval Augmented Generation Systems.” [Online]. Available: <https://arxiv.org/abs/2411.06037>
- [82] A. N. Angelopoulos, S. Bates, C. Fannjiang, M. I. Jordan, and T. Zrnic, “Prediction-Powered Inference.” [Online]. Available: <https://arxiv.org/abs/2301.09633>
- [83] H. Jiang *et al.*, “LongLLMLingua: Accelerating and Enhancing LLMs in Long Context Scenarios via Prompt Compression.” [Online]. Available: <https://arxiv.org/abs/2310.06839>

- [84] F. Xu, W. Shi, and E. Choi, “RECOMP: Improving Retrieval-Augmented LMs with Compression and Selective Augmentation.” [Online]. Available: <https://arxiv.org/abs/2310.04408>
- [85] A. Andoni, P. Indyk, T. Laarhoven, I. Razenshteyn, and L. Schmidt, “Practical and Optimal LSH for Angular Distance.” [Online]. Available: <https://arxiv.org/abs/1509.02897>
- [86] A. Maltsev, “zig-lsh: cross-polytope and hyperplane LSH in Zig.” [Online]. Available: <https://github.com/guanchzhou/zig-lsh>
- [87] J. Su, J. Cao, W. Liu, and Y. Ou, “Whitening Sentence Representations for Better Semantics and Faster Retrieval.” [Online]. Available: <https://arxiv.org/abs/2103.15316>
- [88] W. Webber, A. Moffat, and J. Zobel, “Statistical Power in Retrieval Experimentation,” in *Proceedings of CIKM 2008*, 2008.
- [89] C. Buckley and E. M. Voorhees, “Retrieval Evaluation with Incomplete Information,” in *Proceedings of SIGIR 2004*, 2004.
- [90] P. Thomas, S. Spielman, N. Craswell, and B. Mitra, “Large Language Models Can Accurately Predict Searcher Preferences.” [Online]. Available: <https://arxiv.org/abs/2309.10621>
- [91] T. Chen *et al.*, “Dense X Retrieval: What Retrieval Granularity Should We Use?.” [Online]. Available: <https://arxiv.org/abs/2312.06648>
- [92] R. Miao *et al.*, “GraphER: An Efficient Graph-Based Enrichment and Reranking Method for Retrieval-Augmented Generation.” [Online]. Available: <https://arxiv.org/abs/2603.24925>
- [93] S. Wang *et al.*, “Efficient Algorithms for Approximate Single-Source Personalized PageRank Queries.” [Online]. Available: <https://arxiv.org/abs/1908.10583>
- [94] F. Zhao *et al.*, “Larch: Learned Query Optimization for Semantic Predicates.” [Online]. Available: <https://arxiv.org/abs/2606.07923>
- [95] Y. Jeong, J. Kim, D. Lee, and S.-won Hwang, “ECoRAG: Evidentiality-Guided Compression for Long Context RAG.” [Online]. Available: <https://arxiv.org/abs/2506.05167>
- [96] M. Rainey, U. Acar, and M. Sezer, “SPIRE: Structure-Preserving Interpretable Retrieval of Evidence.” [Online]. Available: <https://arxiv.org/abs/2604.20849>
- [97] RTK contributors, “RTK: a command-line proxy that filters shell output before it reaches a language model.” [Online]. Available: <https://www.rtk-ai.app/>
- [98] A. Gallant, “ripgrep: recursively search directories for a regex pattern.” [Online]. Available: <https://github.com/BurntSushi/ripgrep>
- [99] C. Wohlin, P. Runeson, M. Höst, M. C. Ohlsson, B. Regnell, and A. Wesslén, *Experimentation in Software Engineering*. Berlin, Heidelberg: Springer, 2012. doi: [10.1007/978-3-642-29044-2](https://doi.org/10.1007/978-3-642-29044-2).
- [100] Typst contributors, “Typst: a markup-based typesetting system.” [Online]. Available: <https://github.com/typst/typst>
- [101] National Institute of Standards and Technology, “Secure Hash Standard (SHS),” Federal Information Processing Standards Publication FIPS PUB 180-4, 2015. doi: [10.6028/NIST.FIPS.180-4](https://doi.org/10.6028/NIST.FIPS.180-4).

A Earlier cells

Before the protocol was frozen, two switches were measured on the same development questions with gbrain’s own evaluator. Prefixing each chunk with its note title before embedding did not change recall, MRR, or nDCG beyond the second decimal. gbrain’s local cross-encoder reranker, which scores each question and passage jointly [56], raised MRR and nDCG by about 0.01 and increased latency from 0.13 s to 3.8 s per question.

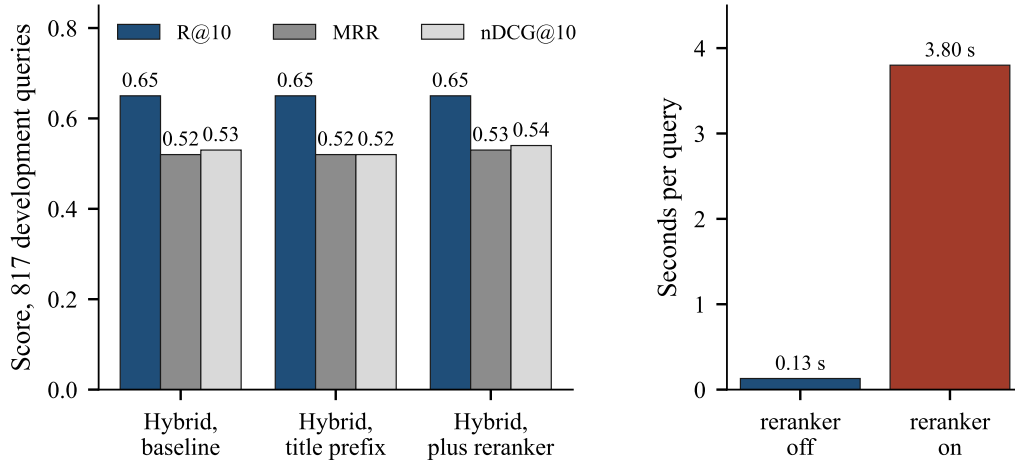


Figure 20: gbrain’s hybrid before and after the title prefix, and with the reranker. Means printed to two decimals by gbrain’s evaluator.

B Harness observations

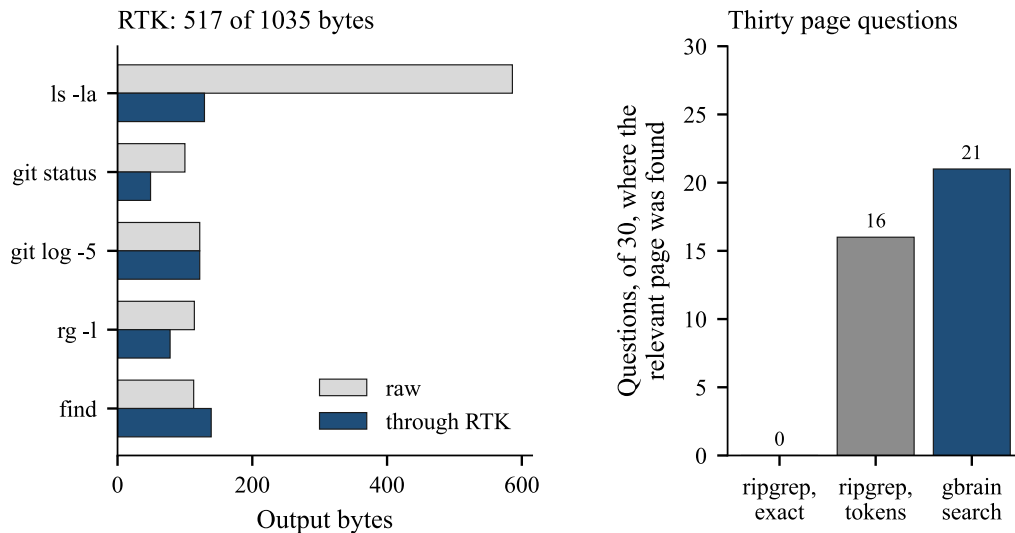


Figure 21: Left: output bytes of five shell commands, raw and through RTK. Right: of thirty development questions, how many found a relevant note by each method.

RTK reduced the combined output from 1,035 to 517 bytes; `ls -la` fell from 586 to 129 bytes, `git log` was unchanged, and `find` grew from 113 to 139. For the page task, thirty development questions (every 27th) were searched by exact ripgrep [98] over the vault, by ripgrep for up to four words longer than five characters, and by gbrain search with a limit of ten; mean time per question was 0.03 s for ripgrep and 8.0 s for gbrain.

C Replication on PostgreSQL 18

After the measurements above, the server was upgraded from PostgreSQL 16.15 with pgvector 0.8.6 to 18.6 with pgvector 0.8.7, and ArangoDB from 3.12.10-1 to 3.12.12. Every measurement that depends on the database software was repeated on the upgraded versions with the same scripts and query vectors (Table 16). This is a replication against stored results, not a side-by-side test. PostgreSQL 16 had been removed by then, the corpus had grown from 5,155 to 5,164 embedded chunks, and each setting ran once under different machine load.

Retrieval quality did not change. Keyword recall at 10 is identical (0.200 for whole notes), and the share of the exact top ten that the HNSW index finds differs by less than 0.001 at every ef_search. Latencies moved in both directions by up to a few milliseconds, which one run per setting cannot attribute to the version. The engine comparison holds. At ef_search 50, pgvector found 0.987 of the exact top ten in 1.21 ms. ArangoDB at nProbe 16 found 0.974 in 4.16 ms. One-hop expansion took 0.16 ms in SQL against 1.13 ms in AQL, with identical neighbour sets for all 300 starts. The 200,000-vector recall is not comparable (marked †). The code of the original in-session run was not kept, so the noisy copies were rebuilt from its description and the data set differs. That script is now part of the repository (bench/pg18/scale_in_session.py).

Table 16: Database-dependent measurements on PostgreSQL 16.15 with pgvector 0.8.6 (stored) and repeated on 18.6 with pgvector 0.8.7; ArangoDB rows on 3.12.10 and 3.12.12. † marks a measure whose data set differs between the runs. Source: bench/pg18/comparison.json.

Group	Measure	16.15	18.6	Unit
pgvector, live chunks	top-10 median	0.458	0.679	ms
	top-10 p95	1.663	1.778	ms
keyword search	keyword-page recall at 10	0.200	0.200	
	keyword-page time, 817 questions	1.970	1.079	s
	keyword-chunk recall at 10	0.138	0.138	
	keyword-chunk time, 817 questions	0.654	0.085	s
hk1 range scans	level 1, 1 ranges, 817 questions	0.017	0.014	s
	level 1, 4 ranges, 817 questions	0.047	0.027	s
	level 1, 8 ranges, 817 questions	0.100	0.045	s
	level 1, 16 ranges, 817 questions	0.101	0.078	s
	level 2, 8 ranges, 817 questions	0.031	0.012	s
	level 2, 16 ranges, 817 questions	0.056	0.020	s
200,000 vectors, one session	top-10 median	0.801	1.124	ms
	top-10 p95	2.933	2.667	ms
	HNSW index size	1,637,187,584	1,637,195,776	bytes
	HNSW build	49.500	68.200	s
200,000 vectors, new psql per query	recall at 10 against exact †	0.588	0.302	
	top-10 median	68.175	28.936	ms
engine comparison, ArangoDB 3.12.10 then 3.12.12	one hk1 prefix range	364.5	88.504	ms
	arango nProbe 1, median	2.004	1.900	ms
	arango nProbe 1, exact top-10 found	0.553	0.564	
	arango nProbe 4, median	3.794	2.772	ms
	arango nProbe 4, exact top-10 found	0.831	0.853	
	arango nProbe 8, median	4.333	2.527	ms
	arango nProbe 8, exact top-10 found	0.921	0.930	
	arango nProbe 16, median	4.603	4.162	ms
	arango nProbe 16, exact top-10 found	0.969	0.974	
	arango nProbe 72, median	4.638	5.259	ms
	arango nProbe 72, exact top-10 found	1.000	1.000	
	arango exact, median	12.621	12.201	ms
	arango exact, exact top-10 found	1.000	1.000	
	pgvector ef_search 50, median	2.098	1.209	ms
engine comparison, pgvector				

Group	Measure	16.15	18.6	Unit
one-hop expansion	pgvector ef_search 50, exact top-10 found	0.986	0.987	
	pgvector ef_search 100, median	2.158	1.587	ms
	pgvector ef_search 100, exact top-10 found	0.994	0.994	
	pgvector ef_search 200, median	1.876	2.197	ms
	pgvector ef_search 200, exact top-10 found	0.999	0.998	
	pgvector ef_search 400, median	2.428	3.033	ms
	pgvector ef_search 400, exact top-10 found	1.000	1.000	
	pgvector exact, median	8.444	11.834	ms
	pgvector exact, exact top-10 found	1.000	1.000	
	AQL median	1.338	1.132	ms
	SQL median	0.453	0.157	ms
	identical neighbour sets	300	300	starts

D Source files

File	Bytes	sha256 (first 16 hex)
arango-dropped.json	135	1d311f869e061f93
arango-hop.json	417	24161014fe573eb2
arango-load.json	558	9b08616dd2cc9002
arango-vector.json	4,232	5f937c6908087b68
arango_bench.py	11,129	1559eb7d61b38909
budget-sweep.json	4,144	6a0b64a4ecf02015
budget_sweep.py	2,117	bb6e61c988165369
centering.json	23,720	5f15bde978b77ad4
centering.py	4,762	e99166b80aa88784
earlier-evals.json	551	57cbf2a4d682261b
engines-d12.json	5,517	7b146467c089fa93
engines_bench.py	14,665	fb532d302fbb50ee
evidence-units.json	1,636	804dd5a19638c67c
h4-ttest.json	951	3e57dcbbd7490610
harness-pages.json	6,010	c1048a81467e13f8
harness_pages.py	2,326	a6f3dbea6fac0689
heldout.json	189	8b94555731413a20
hilbert-sweep.json	10,909	b08475ec9256fa92
hilbert_measure.py	22,804	582aca4068a7b6f6
hk1-efficiency.json	5,902	ca6bec3d2d1c6d2e
hk1_efficiency.py	5,517	052ec546530e909e
hk2-sweep.json	5,733	820348c9efd2e8c5
hk2_measure.py	5,011	beb655d8e69dd08d
investigate.py	18,044	fb3c1886bc2ba10c

File	Bytes	sha256 (first 16 hex)
investigation.json	30,564	0040b2d53587cbea
lsh-qa.json	685	728df8ac87c00120
lsh_qa.py	2,438	92a1cd6aac6d89a2
manifest.json	700	0f293859c7a790d2
metrics.py	2,323	7410909614b1f32b
neighbour-chunk.json	138,517	a46b6aab46ddf72d
neighbour-page.json	84,440	bfd56099d0a90b02
neighbour.py	1,922	38c49f5045efee5f
per-query.json	2,192	28b58d2a7ae68d11
per_query.py	2,874	1d4b7f74ffe481be
pgvector-5143.json	11,990	7344f2a7d2fc84a9
pq-sweep.json	6,329	d767b031ad3a4ca7
pq_measure.py	3,164	6795092377998fe4
rabitq-sweep.json	5,316	7dd703a75e2771dc
rabitq_measure.py	2,948	fa429d0b91004f79
random-filter.json	1,344	101a0a86cd70aa4b
random_filter.py	2,198	4fb4cdaeed346d76
retrieval-grid.json	1,531	2a6b9ff6516b6b2d
rtk-bytes.json	3,771	b56703c9b0049452
run_dev.py	16,030	82b83fc645959dac
run_measure.py	25,123	443471c29b10c968
scale-200k.json	755	e625bc2d8eda8842
scale_200k.py	4,362	812ea909774c08db
section-rule.json	1,136	39eb4cae25286127
spread.json	244	f7e5d030a9cc83e5
spread_heldout.py	3,849	99e4a56d35665a3d
uncertainty.json	2,082	26f205c68cdf4236
uncertainty.py	5,124	d07478e9b1e86094

List 1: Result files and the scripts that wrote them, with the first sixteen hexadecimal digits of each file’s SHA-256 digest [101].

E Index

ArangoDB	14, 15	nDCG	6
BM25	2	pgvector	14
bootstrap interval	7	Postgres	14
gbrain	4	probe level	6
held-out set	18	Qwen3-Embedding	2
Hilbert curve	2	random subset	12
hk1	5	recall at 10	6
HNSW	2	reciprocal rank fusion	2
MRR	6	reranker	39
MUVERA	2	RTK	39

SplitMix64	5
title prefix	39